飞虹 Code

终端 AI 编程智能体软件

源程序代码

软件版本：V8.0.0

著作权人：吴赐虹

开发完成日期：2026年9月3日

首次发表日期：2026年9月3日

源代码页数：前30页 + 后30页，共60页

开发单位：晋江市飞虹智科技企业管理有限公司

研发中心：飞扬企源研发中心

┌── src/cli/version.ts ──┐

```ts
/**
 * 飞虹 Code (对标 Muse Code · 自研内核)
 * 晋江市飞虹智科技企业管理有限公司 · 飞扬企源研发中心 · 负责人：吴赐虹
 *
 * 版本号集中定义
 */
export const VERSION = '8.0.0';
export const PRODUCT = '飞虹 Code';
export const TAGLINE = '终端 AI 编程智能体（对标 Muse Code / Claude Code / Cursor CLI · 自研内核差异化）';
export const SIGNATURE =
  '晋江市飞虹智科技企业管理有限公司 · 飞扬企源研发中心 · 负责人：吴赐虹';

```

┌── src/cli/index.ts ──┐

```ts
#!/usr/bin/env node
/**
 * 飞虹 Code (对标 Muse Code · 自研内核)
 * 晋江市飞虹智科技企业管理有限公司 · 飞扬企源研发中心 · 负责人：吴赐虹
 *
 * CLI 入口：参数解析 → 版本/帮助/单命令/REPL/管理命令 分发
 */
import { randomUUID } from 'crypto';
import { readFileSync, existsSync } from 'fs';
import dns from 'node:dns';
import { setRunId } from '../shared/logger';
import { AppError } from '../shared/errors';
import { loadDotEnv } from '../shared/config';
import { parseArgs, type SkillCommand, type ManagementCommand } from './commands';
import { startRepl } from './repl';
import { runTui } from './tui-run';

// 强制 IPv4 优先：规避 IPv6 链路故障（与 web server 一致，幂等无副作用）。
// 部分网络 IPv6 路由不可达时，默认 verbatim 解析优先走 IPv6 导致 fetch 连接失败。
dns.setDefaultResultOrder('ipv4first');
import {
  runGoal,
  isOfflineByDefault,
  runPlanSkill,
  runGrillSkill,
  runGoalSkill,
  runSelfHealSkill,
  runParallelGoal,
  runSessions,
  runResume,
  runDiff,
  runRollback,
  runWhoami,
  runPolicyCmd,
  runAudit,
  runAuditVerify,
```

```
  runTenants,
  runDoctor,
  runPluginCmd,
  runSkillMarketCmd,
runSkillNewCmd,
  runReviewCmd,
  runTeamCmd,
  runServe,
  runCodeWrite,
  runQualityGate,
  runSelfImprove,
  runSelfEvolve,
  runSwe,
  runModelStats,
  runExperiences,
  runHarness,
} from './run';
import { VERSION } from './version';
import { setLang, t } from '../shared/i18n';

function printVersion(): void {
  console.log(`fhcode v${VERSION}`);
  console.log(`${t('app.product')} — ${t('app.tagline')}`);
  console.log(t('app.signature'));
}

function printHelp(): void {
  console.log(t('cli.help', { version: VERSION, signature: t('app.signature') }));
}

/** M1.3: 读取上下文文件内容（限 32KB；不存在/不可读返回 null） */
const MAX_CONTEXT_BYTES = 32 * 1024;
function readContextFile(path: string): string | null {
  try {
    if (!existsSync(path)) return null;
    const buf = readFileSync(path);
    return buf.subarray(0, MAX_CONTEXT_BYTES).toString('utf8');
  } catch {
    return null;
  }
}

async function main(): Promise<void> {
  setRunId(randomUUID());
  loadDotEnv(); // 尽早加载 .env（须在 isOfflineByDefault / loadConfig 之前）

  const args = parseArgs(process.argv.slice(2));
  setLang(args.flags.lang);

  if (args.flags.version) {
```

```
    printVersion();
    return;
  }
  if (args.flags.help) {
    printHelp();
    return;
  }
  if (args.skill) {
    await dispatchSkill(args.skill);
    return;
  }
  if (args.manage) {
    await dispatchManage(args.manage);
    return;
  }
  if (args.flags.parallel && args.command) {
    await runParallelGoal(args.command);
    return;
  }
  if (args.flags.exec) {
    // -e 直接执行命令（不进入 Agent 编排），等价于在终端里跑一遍
    const { execSync } = await import('child_process');
    try {
      execSync(args.flags.exec, { stdio: 'inherit', shell: true } as unknown as Parameters<typeof execSync
>[1]);
    } catch (e) {
      const code = (e as { status?: number }).status ?? 1;
      process.exitCode = code;
    }
    return;
  }
  if (args.command) {
    const offline = isOfflineByDefault();
    let goal = args.command;
    // M1.3: --context-file 附加活动文件/选区文件内容作为结构化上下文（限 32KB 防爆上下文）
    if (args.flags.contextFile) {
      const content = readContextFile(args.flags.contextFile);
      if (content !== null) {
        goal += `\n\n<context-file: ${args.flags.contextFile}>\n${content}\n</context-file>`;
      }
    }
    await runGoal(goal, { offline, stream: args.flags.stream, model: args.flags.model });
    return;
  }

  await startRepl();
}

async function dispatchSkill(skill: { kind: SkillCommand; arg: string }): Promise<void> {
  if (skill.kind === 'plan') console.log(runPlanSkill(skill.arg || ''));
  else if (skill.kind === 'grill') console.log(runGrillSkill(skill.arg || '.'));
```

```
  else if (skill.kind === 'self-heal') console.log(runSelfHealSkill(skill.arg || ''));
  else console.log(runGoalSkill(skill.arg || ''));
}

async function dispatchManage(m: ManagementCommand): Promise<void> {
  switch (m.kind) {
    case 'sessions': await runSessions(); break;
    case 'resume': await runResume(m.id); break;
    case 'diff': await runDiff(m.id); break;
    case 'rollback': await runRollback(m.id, m.yes); break;
    case 'whoami': runWhoami(); break;
    case 'policy': runPolicyCmd(); break;
    case 'audit': if (m.verify) runAuditVerify(); else runAudit(m.limit); break;
    case 'tenants': runTenants(); break;
    case 'doctor': await runDoctor(); break;
    case 'tui': runTui(); break;
    case 'plugin': await runPluginCmd(m.action, m.source); break;
    case 'skill-market': await runSkillMarketCmd(m.action, m.query, m.market); break;
    case 'skill-new': await runSkillNewCmd(m.name, { template: m.template, global: m.global }); break;
    case 'review': runReviewCmd(m.path, m.json); break;
    case 'team': await runTeamCmd(m.goal); break;
    case 'serve': runServe(m.port); break;
    case 'code-write': runCodeWrite(m.goal); break;
    case 'quality-gate': runQualityGate(m.path); break;
    case 'self-improve': await runSelfImprove(); break;
    case 'self-evolve': await runSelfEvolve(m.action, m.args); break;
    case 'model-stats': runModelStats(); break;
    case 'experiences': runExperiences(m.path); break;
    case 'harness': await runHarness({
      split: m.split,
      limit: m.limit,
      offset: m.offset,
      mode: m.mode,
      verifier: m.verifier,
      testCommand: m.testCommand,
      report: m.report,
      json: m.json,
    }); break;
    case 'swe': await runSwe(m.goal, {
      repo: m.repo,
      maxTasks: m.maxTasks,
      maxRetries: m.maxRetries,
      maxIterations: m.maxIterations,
      verifyOnly: m.verifyOnly,
      planOnly: m.planOnly,
    }); break;
  }
}

main().catch((err: unknown) => {
```

```
  setRunId(randomUUID());
  if (err instanceof AppError) {
    console.error(t('err.runFailed', { code: err.code, message: err.message }));
  } else {
    console.error(t('err.unexpected') + (err instanceof Error ? err.message : String(err)));
  }
  console.error(t('err.hint'));
  process.exit(1);
});

```

┌── src/cli/repl.ts ──┐

```
/**
 * 飞虹 Code (对标 Muse Code · 自研内核)
 * 晋江市飞虹智科技企业管理有限公司 · 飞扬企源研发中心 · 负责人：吴赐虹
 *
 * 交互式 REPL：逐条需求交给编排器执行。未配置模型时走离线模式。
 * v0.5.0 增强：
 *  - Ctrl+D / EOF 优雅退出（此前 question Promise 永不 resolve，进程挂死）
 *  - 支持斜杠技能 /plan /grill /goal /self-heal（与命令行行为一致）
 * P3-1 增强：
 *  - TUI 模式（TTY 时自动启用）：sticky header（模式/runId/迭代/成本/状态）+ 无闪烁渲染
 *  - 编排器事件实时驱动 header 与内容区；非 TTY 环境退化为普通输出
 */
import * as readline from 'readline';
import { runGoal, isOfflineByDefault, runPlanSkill, runGrillSkill, runGoalSkill, runSelfHealSkill } from
'./run';
import type { OrchestratorEvent } from '../agent/orchestrator';
import { Tui } from './tui';
import { t } from '../shared/i18n';

/** 把编排器事件渲染进 TUI，只展示思考内容，不展示工具调用等技术细节 */
function tuiEventRenderer(tui: Tui): (ev: OrchestratorEvent) => void {
  return (ev) => {
    switch (ev.type) {
      case 'model.response':
        if (ev.content.trim()) {
          tui.log(ev.content.trim());
          tui.log('');
        }
        break;
      case 'tool.call':
      case 'tool.result':
        // 工具调用和结果不展示
        break;
      case 'self-heal':
        tui.log('刚才遇到点小问题，我调整一下思路再试试。');
        tui.log('');
        break;
      case 'context.compact':
        // 内部机制，不展示
        break;
```

```
      case 'session.end':
        tui.setStatus({ iteration: ev.iterations, cost: '$' + ev.costUsd.toFixed(6), state: ev.ok ? '完成' : '未完成' });
        break;
    }
  };
}

export async function startRepl(): Promise<void> {
  const tui = new Tui();
  tui.start(); // 非 TTY 自动 no-op

  console.log(t('repl.welcome'));
  console.log(t('repl.hint'));
  tui.setStatus({ mode: isOfflineByDefault() ? 'offline' : 'live', state: t('repl.stateReady') });

  const rl = readline.createInterface({ input: process.stdin, output: process.stdout });

  // Ctrl+D / EOF：readline 触发 close，但挂起的 question 回调不会执行，
  // 必须由 close 事件主动唤醒等待中的 Promise，否则进程永久挂死。
  let closed = false;
  let resolvePrompt: ((line: string | null) => void) | null = null;
  rl.on('close', () => {
    closed = true;
    resolvePrompt?.(null);
  });

  while (!closed) {
    const line = await new Promise<string | null>((resolve) => {
      resolvePrompt = resolve;
      rl.question(t('repl.prompt'), (answer) => resolve(answer));
    });
    resolvePrompt = null;
    if (line === null) break; // EOF → 退出
    const cmd = line.trim();
    if (cmd === 'exit' || cmd === 'quit') break;
    if (!cmd) continue;

    // 斜杠技能（与 CLI 单命令一致）
    if (cmd.startsWith('/')) {
      try {
        const [kind, ...rest] = cmd.slice(1).split(/\s+/);
        if (kind === 'plan') tui.log(runPlanSkill(rest.join(' ') || ''));
        else if (kind === 'grill') tui.log(runGrillSkill(rest.join(' ') || '.'));
        else if (kind === 'goal') tui.log(runGoalSkill(rest.join(' ') || ''));
        else if (kind === 'self-heal') tui.log(runSelfHealSkill(rest.join(' ') || ''));
        else tui.log(t('repl.unknownSkill', { cmd: kind }));
      } catch (e) {
        tui.log(t('repl.errorPrefix') + (e instanceof Error ? e.message : String(e)));
      }
      continue;
```

```
    }

    const offline = isOfflineByDefault();
    tui.setStatus({ mode: offline ? 'offline' : 'live', state: t('repl.stateRunning') });
    try {
      // P3-1：TUI 模式下用自定义渲染器驱动 header/内容；非 TTY 走普通流式输出
      await runGoal(cmd, { offline, renderer: tui.isEnabled ? tuiEventRenderer(tui) : undefined, stream: !tui.isEnabled });
    } catch (e) {
      tui.log(t('repl.errorPrefix') + (e instanceof Error ? e.message : String(e)));
    }
    tui.setStatus({ state: t('repl.stateReady') });
  }

  rl.close();
  tui.stop();
  console.log(t('repl.bye'));
}

┌── src/shared/config.ts ──┐
/**
 * 飞虹 Code (对标 Muse Code · 自研内核)
 * 晋江市飞虹智科技企业管理有限公司 · 飞扬企源研发中心 · 负责人：吴赐虹
 *
 * 集中配置（铁律：所有配置来自环境变量，启动时校验，fail-fast，懒加载）
 */
import { ConfigError } from './errors';
import { readFileSync, existsSync } from 'fs';
import { join } from 'path';
import { homedir } from 'os';
import { VERSION } from '../cli/version';
import type { CapabilityTag, ModelStrategy } from './types';
import type { SandboxMode } from '../tools/sandbox';
import { normalizeSandboxMode } from '../tools/sandbox';
import type { McpServerConfig } from '../tools/mcp/mcp-client';
import { parseMcpServers } from '../tools/mcp';
import type { HookConfig } from '../runtime/hooks';
import { parseHooks } from '../runtime/hooks';
import { loadPlugins, type LoadedPlugins } from '../plugins/plugin-loader';
import { getGithubMcpServers } from '../integrations/github-mcp';

/**
 * 极简 .env 加载器（不引入第三方依赖，离线可用）。
 * 在 cwd 下读取 .env（若存在），将 KEY=VALUE 注入 process.env（仅当该键尚未设置，避免覆盖显式环境变量）。
 * 值两侧的 ' 或 " 会被剥离。.env 必须被 .gitignore 排除，切勿提交真实密钥。
 */
export function loadDotEnv(cwd = process.cwd()): void {
  const file = join(cwd, '.env');
  if (!existsSync(file)) return;
  try {
    const text = readFileSync(file, 'utf8');
```

```
    for (const raw of text.split('\n')) {
      const line = raw.trim();
      if (!line || line.startsWith('#')) continue;
      const eq = line.indexOf('=');
      if (eq === -1) continue;
      const key = line.slice(0, eq).trim();
      let val = line.slice(eq + 1).trim();
      if ((val.startsWith('"') && val.endsWith('"')) || (val.startsWith("'") && val.endsWith("'"))) {
        val = val.slice(1, -1);
      }
      if (!(key in process.env)) process.env[key] = val;
    }
  } catch {
    /* 加载失败不影响离线模式 */
  }
}

export interface ProviderConfig {
  id: string;
  type: 'openai-compatible' | 'ollama';
  baseURL: string;
  /** 模型名（OpenAI 兼容接口必填；Ollama 指定本地模型） */
  model?: string;
  apiKey?: string;
  tags: CapabilityTag[];
  costPer1k?: number;
}

export interface AppConfig {
  app: { name: string; version: string; homeDir: string };
  models: {
    providers: ProviderConfig[];
    defaultStrategy: ModelStrategy;
    budgetPerTaskUsd: number;
  };
  runtime: { logDir: string; maxRetries: number };
  /** P0-3：MCP 服务器列表（外部工具扩展） */
  mcp: { servers: McpServerConfig[] };
  /** P2-1：hooks 确定性控制（PreToolUse/PostToolUse/PostEdit/SessionStart） */
  hooks: HookConfig[];
  /** P3-3：插件（聚合的 skills 目录 / hooks / MCP） */
  plugins: LoadedPlugins;
  security: {
    shellAllowlist: string[];
    requireApproval: boolean;
    /** P0-2：沙箱模式（read-only / workspace-write / danger-full-access） */
    sandboxMode: SandboxMode;
    /** P0-2：网络域名规则（allow/deny，作用于 run_shell 命令中的 http(s) 目标） */
    networkAllow: string[];
    networkDeny: string[];
```

```
  };
}

/**
 * 解析主目录：优先 FH_HOME，缺省 ~/.feihong-code（避免缺环境变量即崩溃）。
 * 支持 `~` 前缀展开（Windows 下 HOME 不一定存在，统一走 os.homedir()）。
 */
export function resolveHomeDir(): string {
  const h = process.env.FH_HOME?.trim();
  if (h) return h.startsWith('~') ? h.replace(/^~/, homedir()) : h;
  return join(homedir(), '.feihong-code');
}

/** 展开路径开头的 `~`（对 FH_LOG_DIR 等单值环境变量复用） */
function expandTilde(p: string): string {
  return p.startsWith('~') ? join(homedir(), p.slice(1)) : p;
}

let cached: AppConfig | null = null;

/**
 * 读取 fhcode 配置文件（JSON），按优先级：
 *   1) 显式 path
 *   2) FH_CONFIG 环境变量
 *   3) cwd/fhcode.config.json
 *   4) FH_HOME/fhcode.config.json
 * 文件缺失或 JSON 损坏时返回 null（不抛错，便于离线/默认配置）。
 */
export function loadConfigFile(path?: string): Partial<AppConfig> | null {
  const candidates = [
    path,
    process.env.FH_CONFIG,
    join(process.cwd(), 'fhcode.config.json'),
    join(resolveHomeDir(), 'fhcode.config.json'),
  ].filter(Boolean) as string[];
  for (const f of candidates) {
    if (!existsSync(f)) continue;
    try {
      return JSON.parse(readFileSync(f, 'utf8')) as Partial<AppConfig>;
    } catch {
      /* 忽略损坏的配置文件 */
    }
  }
  return null;
}

/**
 * 单环境变量快速接入真实模型：
 *   FH_MODEL_NAME         模型名（必填，Ollama 即本地模型名）
 *   FH_MODEL_TYPE         'ollama' | 'openai-compatible'（缺省按 baseURL 推断）
```

```
 *   FH_MODEL_BASE_URL     接口地址（ollama 缺省 http://localhost:11434）
 *   FH_MODEL_API_KEY      OpenAI 兼容接口的 Bearer 令牌
 *   FH_MODEL_TAGS         逗号分隔能力标签（缺省 code-gen,reasoning[,local]）
 *   FH_MODEL_COST_PER_1K  每千 token 成本（USD，统计用，缺省 0）
 * 仅当 FH_PROVIDERS / 配置文件均未提供 provider 时生效。
 */
function buildProviderFromEnv(): ProviderConfig | null {
  const name = process.env.FH_MODEL_NAME || process.env.FH_OLLAMA_MODEL;
  if (!name) return null;
  const type: 'openai-compatible' | 'ollama' =
    (process.env.FH_MODEL_TYPE as 'openai-compatible' | 'ollama') ||
    (process.env.FH_MODEL_BASE_URL?.includes('ollama') ? 'ollama' : 'openai-compatible');
  const baseURL =
    process.env.FH_MODEL_BASE_URL || (type === 'ollama' ? 'http://localhost:11434' : '');
  if (type === 'openai-compatible' && !baseURL) return null;
  const tagStr =
    process.env.FH_MODEL_TAGS ||
    (type === 'ollama' ? 'code-gen,reasoning,local' : 'code-gen,reasoning');
  const tags = tagStr
    .split(',')
    .map((s) => s.trim())
    .filter(Boolean) as CapabilityTag[];
  return {
    id: name,
    type,
    baseURL,
    model: name,
    apiKey: process.env.FH_MODEL_API_KEY || undefined,
    tags,
    costPer1k: Number(process.env.FH_MODEL_COST_PER_1K || '0'),
  };
}

/**
 * 解析模型供应商列表（三级优先级）：
 *   1) FH_PROVIDERS 显式 JSON（最高优先级）
 *   2) 配置文件 models.providers
 *   3) 单环境变量（FH_MODEL_NAME 等）自动构建一个 provider
 * 三者皆无则返回空数组（真实模式会在调用时给出明确报错）。
 */
function resolveProviders(fileCfg?: Partial<AppConfig> | null): ProviderConfig[] {
  if (process.env.FH_PROVIDERS) {
    try {
      const parsed = JSON.parse(process.env.FH_PROVIDERS);
      if (!Array.isArray(parsed)) throw new Error('FH_PROVIDERS 必须为数组');
      return parsed as ProviderConfig[];
    } catch {
      throw new ConfigError('FH_PROVIDERS');
    }
  }
```

```
  const fileProviders = fileCfg?.models?.providers;
  if (Array.isArray(fileProviders) && fileProviders.length > 0) {
    return fileProviders as ProviderConfig[];
  }
  const envProv = buildProviderFromEnv();
  if (envProv) return [envProv];
  return [];
}

/** 加载并校验配置（首次调用时执行，之后复用）。缺必需项立即抛 ConfigError。 */
export function loadConfig(): AppConfig {
  if (cached) return cached;

  const fileCfg = loadConfigFile();
  const providers = resolveProviders(fileCfg);
  // P3-3：插件聚合（用户级 + 项目级）
  const plugins = loadPlugins(process.cwd());

  cached = {
    app: {
      name: 'feihong-code',
      version: VERSION,
      homeDir: resolveHomeDir(),
    },
    models: {
      providers,
      defaultStrategy:
        (process.env.FH_MODEL_STRATEGY as ModelStrategy) ||
        fileCfg?.models?.defaultStrategy ||
        'cost',
      budgetPerTaskUsd: Number(
        process.env.FH_BUDGET_USD || fileCfg?.models?.budgetPerTaskUsd || '0.5',
      ),
    },
    runtime: {
      logDir: process.env.FH_LOG_DIR ? expandTilde(process.env.FH_LOG_DIR) : join(resolveHomeDir(), 'sessi
ons'),
      maxRetries: 3,
    },
    // P0-3：MCP 服务器（FH_MCP_SERVERS 环境变量优先，其次配置文件 mcp.servers；插件 MCP 叠加；GitHub MCP 自动
叠加）
    mcp: {
      servers: [
        ...(parseMcpServers(process.env.FH_MCP_SERVERS).length > 0
          ? parseMcpServers(process.env.FH_MCP_SERVERS)
          : (fileCfg?.mcp?.servers ?? [])),
        ...plugins.mcp,
        ...getGithubMcpServers(fileCfg),
      ],
    },
    // P2-1：hooks（FH_HOOKS 环境变量优先，其次配置文件 hooks；插件 hooks 叠加）
    hooks: [
```

```
      ...(parseHooks(process.env.FH_HOOKS).length > 0
        ? parseHooks(process.env.FH_HOOKS)
        : (fileCfg?.hooks ?? [])),
      ...plugins.hooks,
    ],
    // P3-3：插件（用户级 + 项目级），聚合技能目录/hooks/MCP，与显式配置叠加
    plugins,
    security: {
      shellAllowlist: (process.env.FH_SHELL_ALLOW || '').split(',').map((s) => s.trim()).filter(Boolean),
      requireApproval: process.env.FH_REQUIRE_APPROVAL !== 'false',
      sandboxMode: normalizeSandboxMode(process.env.FH_SANDBOX_MODE),
      networkAllow: (process.env.FH_NETWORK_ALLOW || '').split(',').map((s) => s.trim().toLowerCase()).filter(Boolean),
      networkDeny: (process.env.FH_NETWORK_DENY || '').split(',').map((s) => s.trim().toLowerCase()).filter(Boolean),
    },
  };
  return cached;
}

/** 仅用于测试：重置缓存 */
export function __resetConfigForTest(): void {
  cached = null;
}
```

┌── src/shared/errors.ts ──┐

```
/**
 * 飞虹 Code (对标 Muse Code · 自研内核)
 * 晋江市飞虹智科技企业管理有限公司 · 飞扬企源研发中心 · 负责人：吴赐虹
 *
 * 类型化错误层级（铁律：业务错误必须可分类、可结构化返回）
 */

/** 所有可预期错误的基类 */
export class AppError extends Error {
  constructor(
    message: string,
    public readonly code: string,
    public readonly status: number,
    public readonly isOperational = true,
  ) {
    super(message);
    this.name = this.constructor.name;
  }
}

/** 配置缺失：启动即失败（fail-fast） */
export class ConfigError extends AppError {
  constructor(key: string) {
    super(`缺少必需环境变量: ${key}`, 'CONFIG_ERROR', 500);
  }
}
```

```

/** 模型调用失败 */
export class ModelError extends AppError {
  constructor(
    message: string,
    public readonly provider: string,
    /** 上游 HTTP 状态码（如 429/500/401/400），供路由层按状态码决定轮换/重试策略 */
    public readonly statusCode?: number,
  ) {
    super(`模型调用失败[${provider}]: ${message}`, 'MODEL_ERROR', statusCode ?? 502);
  }
}

/** 工具执行失败 */
export class ToolError extends AppError {
  constructor(tool: string, message: string) {
    super(`工具[${tool}]执行失败: ${message}`, 'TOOL_ERROR', 400);
  }
}

/** 需要人工审批 */
export class ApprovalRequiredError extends AppError {
  constructor(action: string) {
    super(`需人工审批: ${action}`, 'APPROVAL_REQUIRED', 401);
  }
}

/** 安全拦截（危险命令 / 越权） */
export class SecurityError extends AppError {
  constructor(message: string) {
    super(`安全拦截: ${message}`, 'SECURITY_ERROR', 403);
  }
}

/** 类型守卫：判断是否为可预期的业务错误 */
export function isAppError(e: unknown): e is AppError {
  return e instanceof AppError;
}

```

src/shared/logger.ts

```
/**
 * 飞虹 Code (对标 Muse Code · 自研内核)
 * 晋江市飞虹智科技企业管理有限公司 · 飞扬企源研发中心 · 负责人：吴赐虹
 *
 * 结构化 JSON 日志（铁律：禁止散落 console.log，日志带 runId，禁止记录密钥/PII）
 */
import type { LogEntry, RunId } from './types';

let runId: RunId = 'unknown';

```

```
/** 在 CLI 启动时为本次运行设置唯一 runId */
export function setRunId(id: RunId): void {
  runId = id;
}

// M12 修复：扩充敏感 key 识别（覆盖常见密钥/令牌字段名）
const SENSITIVE_KEY_RE =
  /api[_-]?key|apikey|secret|client[_-]?secret|access[_-]?token|refresh[_-]?token|token|password|passwd|pwd|authorization|bearer|private[_-]?key|cookie|credential|passphrase|session[_-]?id|x-api-key/i;
// 值形态：sk-... / JWT / 长十六进制或 base64 令牌（仅对字符串值生效，避免误伤普通文本）
const SENSITIVE_VALUE_RE =
  /\bsk-[A-Za-z0-9_-]{8,}\b|eyJ[A-Za-z0-9_-]+\.[A-Za-z0-9_-]+\.[A-Za-z0-9_-]+|[A-Za-z0-9+/]{32,}={0,2}/;

/** 元数据脱敏：敏感 key 整值遮蔽；普通 key 但值疑似令牌也遮蔽。 */
export function redact(meta: Record<string, unknown>): Record<string, unknown> {
  const out: Record<string, unknown> = {};
  for (const [k, v] of Object.entries(meta)) {
    if (SENSITIVE_KEY_RE.test(k)) {
      out[k] = '[REDACTED]';
    } else if (typeof v === 'string' && SENSITIVE_VALUE_RE.test(v)) {
      out[k] = '[REDACTED]';
    } else {
      out[k] = v;
    }
  }
  return out;
}

function emit(level: LogEntry['level'], msg: string, meta: Record<string, unknown>): void {
  const entry: LogEntry = {
    ts: new Date().toISOString(),
    level,
    msg,
    runId,
    ...redact(meta),
  };
  const line = JSON.stringify(entry);
  if (level === 'error') process.stderr.write(line + '\n');
  else process.stdout.write(line + '\n');
}

export const logger = {
  info: (msg: string, meta: Record<string, unknown> = {}) => emit('info', msg, meta),
  warn: (msg: string, meta: Record<string, unknown> = {}) => emit('warn', msg, meta),
  error: (msg: string, meta: Record<string, unknown> = {}) => emit('error', msg, meta),
  debug: (msg: string, meta: Record<string, unknown> = {}) => emit('debug', msg, meta),
};

┌── src/shared/secure-store.ts ──┐
/**
 * 飞虹 Code - 安全存储与通信加密工具
```

```
 * 晋江市飞虹智科技企业管理有限公司 · 飞扬企源研发中心 · 负责人：吴赐虹
 *
 * 三重加密体系支撑模块：
 *  - AES-256-GCM：敏感数据落盘加密（模型 API Key、会话令牌）【存储层】
 *  - RSA-2048：敏感参数端到端加密传输（App 公钥加密 → 服务器私钥解密）【通信层】
 *  - 密钥管理：主密钥优先取环境变量 FH_SECRET，否则首次生成持久化 $FH_HOME/.secret
 *    （RSA 私钥持久化 $FH_HOME/rsa_private.pem，公钥经 API 提供给客户端）
 */
import {
  createCipheriv,
  createDecipheriv,
  createHash,
  randomBytes,
  generateKeyPairSync,
  publicEncrypt,
  privateDecrypt,
  constants,
} from 'crypto';
import { existsSync, readFileSync, writeFileSync, mkdirSync } from 'fs';
import { join } from 'path';

/** 主密钥：优先环境变量 FH_SECRET（>=16 字符），否则首次生成并持久化 $FH_HOME/.secret */
export function getMasterKey(homeDir: string): Buffer {
  const env = process.env.FH_SECRET;
  if (env && env.length >= 16) return createHash('sha256').update(env).digest();
  const file = join(homeDir, '.secret');
  if (existsSync(file)) {
    const hex = readFileSync(file, 'utf8').trim();
    if (hex.length === 64) return Buffer.from(hex, 'hex');
  }
  try {
    mkdirSync(homeDir, { recursive: true });
    const key = randomBytes(32);
    writeFileSync(file, key.toString('hex'), 'utf8');
    return key;
  } catch {
    // 极端情况（目录不可写）：退回进程内随机密钥（重启失效，仅兜底）
    return randomBytes(32);
  }
}

/** AES-256-GCM 加密，输出格式 v1:iv:cipher:tag（全部 base64） */
export function encryptText(plain: string, key: Buffer): string {
  const iv = randomBytes(12);
  const cipher = createCipheriv('aes-256-gcm', key, iv);
  const enc = Buffer.concat([cipher.update(String(plain), 'utf8'), cipher.final()]);
  const tag = cipher.getAuthTag();
  return `v1:${iv.toString('base64')}:${enc.toString('base64')}:${tag.toString('base64')}`;
}

```

```
/** AES-256-GCM 解密；格式错误/密钥错误返回空串（调用方按空处理） */
export function decryptText(payload: string, key: Buffer): string {
  try {
    const parts = String(payload).split(':');
    if (parts.length !== 4 || parts[0] !== 'v1') return '';
    const iv = Buffer.from(parts[1], 'base64');
    const enc = Buffer.from(parts[2], 'base64');
    const tag = Buffer.from(parts[3], 'base64');
    const decipher = createDecipheriv('aes-256-gcm', key, iv);
    decipher.setAuthTag(tag);
    return Buffer.concat([decipher.update(enc), decipher.final()]).toString('utf8');
  } catch {
    return '';
  }
}

/** 是否已是加密格式 */
export function isEncrypted(s: unknown): boolean {
  return typeof s === 'string' && s.startsWith('v1:');
}

/** RSA-2048 密钥对：首次生成并持久化，后续复用；读取到空/无效密钥时自动重建 */
export function getRsaKeys(homeDir: string): { publicKey: string; privateKey: string } {
  const pubFile = join(homeDir, 'rsa_public.pem');
  const prvFile = join(homeDir, 'rsa_private.pem');
  if (existsSync(pubFile) && existsSync(prvFile)) {
    try {
      const pub = readFileSync(pubFile, 'utf8');
      const prv = readFileSync(prvFile, 'utf8');
      // 防御性校验：密钥必须包含 PEM 标记且非空，否则视为损坏，重新生成
      if (pub && pub.includes('BEGIN PUBLIC KEY') && prv && prv.includes('BEGIN PRIVATE KEY')) {
        return { publicKey: pub, privateKey: prv };
      }
    } catch {
      // 读取失败，走重新生成逻辑
    }
  }
  try {
    mkdirSync(homeDir, { recursive: true });
    const { publicKey, privateKey } = generateKeyPairSync('rsa', {
      modulusLength: 2048,
      publicKeyEncoding: { type: 'spki', format: 'pem' },
      privateKeyEncoding: { type: 'pkcs8', format: 'pem' },
    });
    writeFileSync(pubFile, publicKey, 'utf8');
    writeFileSync(prvFile, privateKey, 'utf8');
    return { publicKey, privateKey };
  } catch {
    const { publicKey, privateKey } = generateKeyPairSync('rsa', {
      modulusLength: 2048,
```

```
      publicKeyEncoding: { type: 'spki', format: 'pem' },
      privateKeyEncoding: { type: 'pkcs8', format: 'pem' },
    });
    return { publicKey, privateKey };
  }
}

/** RSA 公钥加密（OAEP-SHA256），返回 base64 */
export function rsaEncrypt(plain: string, publicKeyPem: string): string {
  return publicEncrypt(
    { key: publicKeyPem, padding: constants.RSA_PKCS1_OAEP_PADDING, oaepHash: 'sha256' },
    Buffer.from(String(plain), 'utf8'),
  ).toString('base64');
}

/** RSA 私钥解密（OAEP-SHA256）；失败返回空串 */
export function rsaDecrypt(cipherB64: string, privateKeyPem: string): string {
  try {
    return privateDecrypt(
      { key: privateKeyPem, padding: constants.RSA_PKCS1_OAEP_PADDING, oaepHash: 'sha256' },
      Buffer.from(String(cipherB64), 'base64'),
    ).toString('utf8');
  } catch {
    return '';
  }
}

┌── src/shared/sqlite-store.ts ──┐
/**
 * 飞虹 Code — SQLite 数据存储模块（v8.0）
 * 晋江市飞虹智科技企业管理有限公司 · 飞扬企源研发中心 · 负责人：吴赐虹
 *
 * 基于 Node.js 内置 node:sqlite（DatabaseSync），零外部依赖。
 * 统一替代文件/JSON 存储，提供：
 *   - 通用 KV 存储（models/config/memory 等）
 *   - 任务表（替代 task-queue 的 <id>.json 文件）
 *   - 记忆表（MEMORY.md / USER.md / 历史摘要）
 *   - 技能表（Skills）
 *   - 知识库表（Knowledge docs）
 *   - 会话/用户建模（Honcho 风格）
 *
 * 设计：
 *   - 单例连接，路径默认 $FH_HOME/feihong.db（可 FH_DB_PATH 覆盖）
 *   - 启动自动建表 + 迁移（SCHEMA_VERSION 版本号递增）
 *   - 所有写操作 try/catch + 回滚保护
 *   - 敏感字段按表声明加密（复用 secure-store 的 AES-256-GCM）
 */

import { DatabaseSync } from 'node:sqlite';
import { existsSync, mkdirSync } from 'fs';
```

```
import { join } from 'path';
import { getMasterKey, encryptText, decryptText } from './secure-store';

/** 当前 Schema 版本：每次变更结构需 +1 并补 migrate() 分支 */
export const SCHEMA_VERSION = 1;

/** 需要加密的字段（key 表名:字段名） */
const ENCRYPTED_FIELDS: Record<string, string[]> = {
  'config': ['value'],
  'models': ['apiKey'],
};

export interface SQLiteStoreOptions {
  /** 数据库文件路径（默认 $FH_HOME/feihong.db） */
  dbPath?: string;
  /** 是否输出调试日志 */
  debug?: boolean;
}

interface Row {
  [key: string]: unknown;
}

function resolveDbPath(): string {
  if (process.env.FH_DB_PATH) return process.env.FH_DB_PATH;
  const home = process.env.FH_HOME || join(process.env.HOME || process.env.USERPROFILE || '.', '.feihong-code');
  return join(home, 'feihong.db');
}

export class SQLiteStore {
  private db: DatabaseSync;
  private debugEnabled: boolean;
  private migrations: Record<number, (db: DatabaseSync) => void>;

  constructor(options: SQLiteStoreOptions = {}) {
    const dbPath = options.dbPath || resolveDbPath();
    this.debugEnabled = options.debug || false;

    // 确保目录存在
    const dir = dbPath.substring(0, dbPath.lastIndexOf(/[\\/]/.exec(dbPath)?.[0] || '/'));
    if (dir && !existsSync(dir)) mkdirSync(dir, { recursive: true });

    this.db = new DatabaseSync(dbPath);
    this.db.exec('PRAGMA journal_mode = WAL;');
    this.db.exec('PRAGMA foreign_keys = ON;');

    // 迁移定义：key = schema 版本
    this.migrations = {
      1: (db) => this.migrateToV1(db),
    };
```

```
901
902     this.ensureSchema();
903   }
904
905   /* ========== 私有方法 ========== */
906
907   private debug(...args: unknown[]): void {
908     if (this.debugEnabled) console.log('[SQLiteStore]', ...args);
909   }
910
911   private encrypt(table: string, field: string, value: string): string {
912     if (!value) return value;
913     const encFields = ENCRYPTED_FIELDS[table];
914     if (!encFields || !encFields.includes(field)) return value;
915     try {
916       const key = getMasterKey(process.env.FH_HOME || join(process.env.HOME || '.', '.feihong-code'));
917       return encryptText(value, key);
918     } catch {
919       return value;
920     }
921   }
922
923   private decrypt(table: string, field: string, value: string): string {
924     if (!value) return value;
925     const encFields = ENCRYPTED_FIELDS[table];
926     if (!encFields || !encFields.includes(field)) return value;
927     if (!value.startsWith('v1:')) return value; // 未加密的旧数据
928     try {
929       const key = getMasterKey(process.env.FH_HOME || join(process.env.HOME || '.', '.feihong-code'));
930       return decryptText(value, key);
931     } catch {
932       return value;
933     }
934   }
935
936   /* ========== Schema 管理 ========== */
937
938   private migrateToV1(db: DatabaseSync): void {
939     // 元数据表
940     db.exec(`
941       CREATE TABLE IF NOT EXISTS meta (
942         key TEXT PRIMARY KEY,
943         value TEXT
944       );
945
946       -- 通用 KV 存储（models/config/settings 等）
947       CREATE TABLE IF NOT EXISTS kv (
948         namespace TEXT NOT NULL,
949         key TEXT NOT NULL,
950         value TEXT NOT NULL,
```

```
951           updated_at INTEGER NOT NULL DEFAULT (unixepoch()),
952           PRIMARY KEY (namespace, key)
953         );
954
955         -- 任务表
956         CREATE TABLE IF NOT EXISTS tasks (
957           id TEXT PRIMARY KEY,
958           type TEXT NOT NULL,
959           status TEXT NOT NULL DEFAULT 'pending',
960           progress INTEGER NOT NULL DEFAULT 0,
961           input TEXT,
962           result TEXT,
963           error TEXT,
964           model TEXT,
965           created_at INTEGER NOT NULL DEFAULT (unixepoch()),
966           started_at INTEGER,
967           completed_at INTEGER
968         );
969         CREATE INDEX IF NOT EXISTS idx_tasks_status ON tasks(status);
970         CREATE INDEX IF NOT EXISTS idx_tasks_created ON tasks(created_at);
971
972         -- 模型表
973         CREATE TABLE IF NOT EXISTS models (
974           id TEXT PRIMARY KEY,
975           name TEXT NOT NULL,
976           provider TEXT NOT NULL DEFAULT 'openai-compatible',
977           api_base TEXT,
978           api_key TEXT,
979           context_window INTEGER,
980           max_output INTEGER,
981           supports_streaming INTEGER NOT NULL DEFAULT 1,
982           supports_vision INTEGER NOT NULL DEFAULT 0,
983           created_at INTEGER NOT NULL DEFAULT (unixepoch())
984         );
985
986         -- Agent 表
987         CREATE TABLE IF NOT EXISTS agents (
988           id TEXT PRIMARY KEY,
989           name TEXT NOT NULL,
990           type TEXT NOT NULL DEFAULT 'solo',
991           status TEXT NOT NULL DEFAULT 'idle',
992           system_prompt TEXT,
993           model TEXT,
994           created_at INTEGER NOT NULL DEFAULT (unixepoch()),
995           last_active INTEGER
996         );
997
998         -- 技能表
999         CREATE TABLE IF NOT EXISTS skills (
1000          id TEXT PRIMARY KEY,
```

```
  name TEXT NOT NULL,
  description TEXT,
  trigger TEXT,
  prompt TEXT,
  tags TEXT,
  version TEXT DEFAULT '1.0.0',
  use_count INTEGER NOT NULL DEFAULT 0,
  builtin INTEGER NOT NULL DEFAULT 0,
  created_at INTEGER NOT NULL DEFAULT (unixepoch())
);
CREATE INDEX IF NOT EXISTS idx_skills_name ON skills(name);

-- 记忆表（MEMORY.md / USER.md / 摘要）
CREATE TABLE IF NOT EXISTS memory (
  key TEXT PRIMARY KEY,
  content TEXT NOT NULL,
  kind TEXT NOT NULL DEFAULT 'note',
  updated_at INTEGER NOT NULL DEFAULT (unixepoch())
);

-- 记忆摘要/历史（Honcho 风格）
CREATE TABLE IF NOT EXISTS memory_history (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  session_id TEXT,
  summary TEXT NOT NULL,
  type TEXT,
  created_at INTEGER NOT NULL DEFAULT (unixepoch())
);
CREATE INDEX IF NOT EXISTS idx_memory_history_created ON memory_history(created_at);

-- 用户建模（Honcho 风格）
CREATE TABLE IF NOT EXISTS users (
  id TEXT PRIMARY KEY,
  name TEXT,
  preferences TEXT,
  traits TEXT,
  created_at INTEGER NOT NULL DEFAULT (unixepoch()),
  updated_at INTEGER NOT NULL DEFAULT (unixepoch())
);

-- 用户记忆条目
CREATE TABLE IF NOT EXISTS user_memory (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  user_id TEXT NOT NULL,
  content TEXT NOT NULL,
  category TEXT NOT NULL DEFAULT 'fact',
  importance REAL NOT NULL DEFAULT 0.5,
  created_at INTEGER NOT NULL DEFAULT (unixepoch()),
  FOREIGN KEY (user_id) REFERENCES users(id)
);
```

```
      CREATE INDEX IF NOT EXISTS idx_user_memory_user ON user_memory(user_id);

      -- 知识库文档表
      CREATE TABLE IF NOT EXISTS knowledge (
        id TEXT PRIMARY KEY,
        title TEXT NOT NULL,
        content TEXT,
        type TEXT DEFAULT 'markdown',
        tags TEXT,
        size INTEGER,
        created_at INTEGER NOT NULL DEFAULT (unixepoch())
      );
      CREATE INDEX IF NOT EXISTS idx_knowledge_title ON knowledge(title);
    `);

    db.prepare('INSERT OR REPLACE INTO meta(key, value) VALUES(?, ?)').run('schema_version', String(SCHEMA_VERSION));
  }

  private ensureSchema(): void {
    // 先检查 meta 表是否存在，判断是否全新库
    try {
      const metaTable = this.db.prepare(
        "SELECT name FROM sqlite_master WHERE type='table' AND name='meta'"
      ).get() as Row | undefined;

      if (!metaTable) {
        // 全新库：直接建 V1（migrateToV1 内部会建 meta 表并写入版本）
        this.migrateToV1(this.db);
        this.debug('全新数据库初始化到 schema v1');
        return;
      }

      // 已有库：读取版本号，按版本递增迁移
      const row = this.db.prepare("SELECT value FROM meta WHERE key = 'schema_version'").get() as Row | undefined;
      const currentVersion = row ? parseInt(String(row.value), 10) : 0;

      for (let v = currentVersion + 1; v <= SCHEMA_VERSION; v++) {
        if (this.migrations[v]) {
          this.migrations[v](this.db);
          this.db.prepare('INSERT OR REPLACE INTO meta(key, value) VALUES(?, ?)').run('schema_version', String(v));
          this.debug(`迁移到 schema v${v}`);
        }
      }
    } catch (e) {
      console.error('[SQLiteStore] schema 初始化失败:', e);
      throw e;
    }
  }

  /* ========== 通用 KV 存储 ========== */
```

```
1101
1102    kvSet(namespace: string, key: string, value: unknown): void {
1103      const v = typeof value === 'string' ? value : JSON.stringify(value);
1104      this.db.prepare(
1105        'INSERT INTO kv(namespace, key, value, updated_at) VALUES(?, ?, ?, unixepoch()) ' +
1106        'ON CONFLICT(namespace, key) DO UPDATE SET value = excluded.value, updated_at = unixepoch()'
1107      ).run(namespace, key, this.encrypt('config', 'value', v));
1108    }
1109
1110    kvGet<T = unknown>(namespace: string, key: string, defaultValue?: T): T | undefined {
1111      const row = this.db.prepare('SELECT value FROM kv WHERE namespace = ? AND key = ?').get(namespace, ke
     y) as Row | undefined;
1112      if (!row) return defaultValue;
1113      const raw = this.decrypt('config', 'value', String(row.value));
1114      try { return JSON.parse(raw) as T; } catch { return raw as unknown as T; }
1115    }
1116
1117    kvDelete(namespace: string, key: string): void {
1118      this.db.prepare('DELETE FROM kv WHERE namespace = ? AND key = ?').run(namespace, key);
1119    }
1120
1121    kvList(namespace: string): Array<{ key: string; value: unknown }> {
1122      const rows = this.db.prepare('SELECT key, value FROM kv WHERE namespace = ? ORDER BY updated_at DES
     C').all(namespace) as Array<Row>;
1123      return rows.map((r) => {
1124        const raw = this.decrypt('config', 'value', String(r.value));
1125        let value: unknown;
1126        try { value = JSON.parse(raw); } catch { value = raw; }
1127        return { key: String(r.key), value };
1128      });
1129    }
1130
1131    /* ========== 任务 CRUD ========== */
1132
1133    taskUpsert(task: {
1134      id: string; type: string; status: string; progress?: number;
1135      input?: unknown; result?: unknown; error?: string; model?: string;
1136    }): void {
1137      this.db.prepare(
1138        `INSERT INTO tasks(id, type, status, progress, input, result, error, model, created_at, started_at,
     completed_at)
1139         VALUES(?, ?, ?, ?, ?, ?, ?, ?, unixepoch(), ?, ?)
1140         ON CONFLICT(id) DO UPDATE SET
1141           status = excluded.status,
1142           progress = excluded.progress,
1143           input = excluded.input,
1144           result = excluded.result,
1145           error = excluded.error,
1146           model = excluded.model,
1147           started_at = COALESCE(excluded.started_at, tasks.started_at),
1148           completed_at = excluded.completed_at`
1149      ).run(
1150        task.id, task.type, task.status, task.progress ?? 0,
```

```
      task.input !== undefined ? JSON.stringify(task.input) : null,
      task.result !== undefined ? JSON.stringify(task.result) : null,
      task.error ?? null, task.model ?? null,
      task.status === 'running' ? Date.now() : null,
      task.status === 'completed' || task.status === 'failed' ? Date.now() : null
    );
  }

  taskGet(id: string): Row | undefined {
    const row = this.db.prepare('SELECT * FROM tasks WHERE id = ?').get(id) as Row | undefined;
    if (row) {
      if (row.input) row.input = JSON.parse(String(row.input));
      if (row.result) row.result = JSON.parse(String(row.result));
    }
    return row;
  }

  taskList(options: { status?: string; limit?: number; offset?: number } = {}): Row[] {
    const limit = options.limit || 20;
    const offset = options.offset || 0;
    if (options.status) {
      return this.db.prepare(
        'SELECT * FROM tasks WHERE status = ? ORDER BY created_at DESC LIMIT ? OFFSET ?'
      ).all(options.status, limit, offset) as Row[];
    }
    return this.db.prepare('SELECT * FROM tasks ORDER BY created_at DESC LIMIT ? OFFSET ?').all(limit, off
set) as Row[];
  }

  taskCount(): { total: number; pending: number; running: number } {
    const total = (this.db.prepare('SELECT COUNT(*) as c FROM tasks').get() as Row).c as number;
    const pending = (this.db.prepare("SELECT COUNT(*) as c FROM tasks WHERE status = 'pending'").get() as
Row).c as number;
    const running = (this.db.prepare("SELECT COUNT(*) as c FROM tasks WHERE status = 'running'").get() as
Row).c as number;
    return { total, pending, running };
  }

  taskDelete(id: string): void {
    this.db.prepare('DELETE FROM tasks WHERE id = ?').run(id);
  }

  /* ========== 模型 CRUD ========== */

  modelUpsert(model: {
    id: string; name: string; provider?: string; apiBase?: string;
    apiKey?: string; contextWindow?: number; maxOutput?: number;
    supportsStreaming?: boolean; supportsVision?: boolean;
  }): void {
    this.db.prepare(
      `INSERT INTO models(id, name, provider, api_base, api_key, context_window, max_output, supports_stre
aming, supports_vision)
       VALUES(?, ?, ?, ?, ?, ?, ?, ?, ?)
       ON CONFLICT(id) DO UPDATE SET
```

```
        name = excluded.name, provider = excluded.provider, api_base = excluded.api_base,
        api_key = excluded.api_key, context_window = excluded.context_window,
        max_output = excluded.max_output, supports_streaming = excluded.supports_streaming,
        supports_vision = excluded.supports_vision`
    ).run(
      model.id, model.name, model.provider || 'openai-compatible', model.apiBase || null,
      this.encrypt('models', 'apiKey', model.apiKey || ''),
      model.contextWindow ?? null, model.maxOutput ?? null,
      model.supportsStreaming !== false ? 1 : 0,
      model.supportsVision ? 1 : 0
    );
  }

  modelList(): Row[] {
    const rows = this.db.prepare('SELECT * FROM models ORDER BY created_at').all() as Row[];
    return rows.map((r) => ({
      ...r,
      apiKey: this.decrypt('models', 'apiKey', String(r.api_key || '')),
    }));
  }

  modelGet(id: string): Row | undefined {
    const row = this.db.prepare('SELECT * FROM models WHERE id = ?').get(id) as Row | undefined;
    if (row) row.apiKey = this.decrypt('models', 'apiKey', String(row.api_key || ''));
    return row;
  }

  /** 返回数据库原始行（api_key 保持加密态），供安全审计/验证使用 */
  modelGetRaw(id: string): Row | undefined {
    return this.db.prepare('SELECT * FROM models WHERE id = ?').get(id) as Row | undefined;
  }

  modelDelete(id: string): void {
    this.db.prepare('DELETE FROM models WHERE id = ?').run(id);
  }

  /* ========== 技能 CRUD ========== */

  skillUpsert(skill: {
    id: string; name: string; description?: string; trigger?: string;
    prompt?: string; tags?: string[]; version?: string; useCount?: number; builtin?: boolean;
  }): void {
    this.db.prepare(
      `INSERT INTO skills(id, name, description, trigger, prompt, tags, version, use_count, builtin)
       VALUES(?, ?, ?, ?, ?, ?, ?, ?, ?)
       ON CONFLICT(id) DO UPDATE SET
         name = excluded.name, description = excluded.description, trigger = excluded.trigger,
         prompt = excluded.prompt, tags = excluded.tags, version = excluded.version,
         use_count = excluded.use_count, builtin = excluded.builtin`
    ).run(
```

```
      skill.id, skill.name, skill.description || null, skill.trigger || null,
      skill.prompt || null, skill.tags ? JSON.stringify(skill.tags) : null,
      skill.version || '1.0.0', skill.useCount || 0, skill.builtin ? 1 : 0
    );
  }

  skillList(): Row[] {
    const rows = this.db.prepare('SELECT * FROM skills ORDER BY created_at').all() as Row[];
    return rows.map((r) => ({ ...r, tags: r.tags ? JSON.parse(String(r.tags)) : [] }));
  }

  skillSearch(query: string): Row[] {
    const q = `%${query}%`;
    return this.db.prepare(
      'SELECT * FROM skills WHERE name LIKE ? OR description LIKE ? OR trigger LIKE ? ORDER BY use_count DESC'
    ).all(q, q, q) as Row[];
  }

  skillDelete(id: string): void {
    this.db.prepare('DELETE FROM skills WHERE id = ?').run(id);
  }

  /* ========== 记忆 CRUD ========== */

  memorySet(key: string, content: string, kind = 'note'): void {
    this.db.prepare(
      'INSERT INTO memory(key, content, kind, updated_at) VALUES(?, ?, ?, unixepoch()) ' +
      'ON CONFLICT(key) DO UPDATE SET content = excluded.content, kind = excluded.kind, updated_at = unixepoch()'
    ).run(key, content, kind);
  }

  memoryGet(key: string): string | undefined {
    const row = this.db.prepare('SELECT content FROM memory WHERE key = ?').get(key) as Row | undefined;
    return row ? String(row.content) : undefined;
  }

  memoryList(kind?: string): Row[] {
    if (kind) return this.db.prepare('SELECT * FROM memory WHERE kind = ? ORDER BY updated_at DESC').all(kind) as Row[];
    return this.db.prepare('SELECT * FROM memory ORDER BY updated_at DESC').all() as Row[];
  }

  memoryAddHistory(sessionId: string | null, summary: string, type?: string): void {
    this.db.prepare('INSERT INTO memory_history(session_id, summary, type) VALUES(?, ?, ?)').run(sessionId, summary, type || null);
  }

  memoryRecall(keyword: string, limit = 5): Row[] {
    const q = `%${keyword}%`;
    return this.db.prepare(
      'SELECT * FROM memory_history WHERE summary LIKE ? ORDER BY created_at DESC LIMIT ?'
    ).all(q, limit) as Row[];
```

```
  }

  /* ========== 用户建模（Honcho 风格） ========== */

  userUpsert(user: {
    id: string; name?: string; preferences?: unknown; traits?: unknown;
  }): void {
    this.db.prepare(
      `INSERT INTO users(id, name, preferences, traits, created_at, updated_at)
       VALUES(?, ?, ?, ?, unixepoch(), unixepoch())
       ON CONFLICT(id) DO UPDATE SET
         name = excluded.name, preferences = excluded.preferences,
         traits = excluded.traits, updated_at = unixepoch()`
    ).run(
      user.id, user.name || null,
      user.preferences !== undefined ? JSON.stringify(user.preferences) : null,
      user.traits !== undefined ? JSON.stringify(user.traits) : null
    );
  }

  userGet(id: string): Row | undefined {
    const row = this.db.prepare('SELECT * FROM users WHERE id = ?').get(id) as Row | undefined;
    if (row) {
      if (row.preferences) row.preferences = JSON.parse(String(row.preferences));
      if (row.traits) row.traits = JSON.parse(String(row.traits));
    }
    return row;
  }

  userAddMemory(userId: string, content: string, category = 'fact', importance = 0.5): void {
    this.db.prepare('INSERT INTO user_memory(user_id, content, category, importance) VALUES(?, ?, ?, ?)')
      .run(userId, content, category, importance);
  }

  userListMemory(userId: string, limit = 20): Row[] {
    return this.db.prepare(
      'SELECT * FROM user_memory WHERE user_id = ? ORDER BY importance DESC, created_at DESC LIMIT ?'
    ).all(userId, limit) as Row[];
  }

  userSearchMemory(userId: string, keyword: string, limit = 10): Row[] {
    const q = `%${keyword}%`;
    return this.db.prepare(
      'SELECT * FROM user_memory WHERE user_id = ? AND content LIKE ? ORDER BY importance DESC LIMIT ?'
    ).all(userId, q, limit) as Row[];
  }

  /* ========== 知识库 CRUD ========== */

  knowledgeUpsert(doc: {
```

```
    id: string; title: string; content?: string; type?: string; tags?: string[]; size?: number;
  }): void {
    this.db.prepare(
      `INSERT INTO knowledge(id, title, content, type, tags, size, created_at)
       VALUES(?, ?, ?, ?, ?, ?, unixepoch())
       ON CONFLICT(id) DO UPDATE SET
         title = excluded.title, content = excluded.content, type = excluded.type,
         tags = excluded.tags, size = excluded.size`
    ).run(
      doc.id, doc.title, doc.content || null, doc.type || 'markdown',
      doc.tags ? JSON.stringify(doc.tags) : null, doc.size ?? (doc.content?.length || 0)
    );
  }

  knowledgeList(): Row[] {
    const rows = this.db.prepare('SELECT id, title, type, tags, size, created_at FROM knowledge ORDER BY c
reated_at').all() as Row[];
    return rows.map((r) => ({ ...r, tags: r.tags ? JSON.parse(String(r.tags)) : [] }));
  }

  knowledgeSearch(query: string): Row[] {
    const q = `%${query}%`;
    return this.db.prepare(
      'SELECT * FROM knowledge WHERE title LIKE ? OR content LIKE ? ORDER BY created_at DESC'
    ).all(q, q) as Row[];
  }

  knowledgeDelete(id: string): void {
    this.db.prepare('DELETE FROM knowledge WHERE id = ?').run(id);
  }

  /* ========== 统计与健康 ========== */

  stats(): Record<string, number> {
    const count = (table: string): number => {
      try { return (this.db.prepare(`SELECT COUNT(*) as c FROM ${table}`).get() as Row).c as number; }
      catch { return 0; }
    };
    return {
      tasks: count('tasks'),
      models: count('models'),
      agents: count('agents'),
      skills: count('skills'),
      memory: count('memory'),
      memoryHistory: count('memory_history'),
      users: count('users'),
      userMemory: count('user_memory'),
      knowledge: count('knowledge'),
      schemaVersion: SCHEMA_VERSION,
    };
  }
```

```
1401
1402    /** 健康检查：执行简单查询验证数据库可访问 */
1403    health(): { ok: boolean; backend: string; version: number; dbFile: string } {
1404      try {
1405        const row = this.db.prepare("SELECT value FROM meta WHERE key = 'schema_version'").get() as Row;
1406        return {
1407          ok: true,
1408          backend: 'sqlite',
1409          version: parseInt(String(row.value), 10),
1410          dbFile: this.dbFile(),
1411        };
1412      } catch (e) {
1413        return { ok: false, backend: 'error', version: 0, dbFile: this.dbFile() };
1414      }
1415    }
1416
1417    dbFile(): string {
1418      // DatabaseSync 不直接暴露路径，通过环境变量或默认推断
1419      return process.env.FH_DB_PATH || join(process.env.FH_HOME || join(process.env.HOME || '.', '.feihong-c
ode'), 'feihong.db');
1420    }
1421
1422    /** 关闭数据库连接 */
1423    close(): void {
1424      try { this.db.close(); } catch { /* 已关闭 */ }
1425    }
1426  }
1427
1428  /** 全局单例 */
1429  let storeInstance: SQLiteStore | null = null;
1430
1431  export function getStore(options?: SQLiteStoreOptions): SQLiteStore {
1432    if (!storeInstance) storeInstance = new SQLiteStore(options);
1433    return storeInstance;
1434  }
1435
1436  export function resetStore(): void {
1437    if (storeInstance) { storeInstance.close(); storeInstance = null; }
1438  }
1439
┌── src/shared/log-rotator.ts ──┐
1440  /**
1441   * 飞虹 Code — 日志文件轮转工具（独立模块，不影响原有 logger）
1442   * 版本：v7.9.1
1443   *
1444   * 功能：
1445   * - 按大小轮转（默认 10MB）
1446   * - 按时间轮转（默认每天）
1447   * - 自动清理旧日志（保留最近 N 个文件或 N 天）
1448   * - 可与原有结构化 JSON logger 配合使用
1449   *
```

```
 * 用法：
 *   import { LogRotator } from './log-rotator';
 *   const rotator = new LogRotator({ dir: './logs', maxSize: '10m', maxFiles: 10 });
 *   rotator.write('日志内容');
 *
 * 或作为 stream 使用：
 *   const stream = rotator.createWriteStream();
 *   stream.write('日志内容\n');
 */

import * as fs from 'fs';
import * as path from 'path';

export interface LogRotatorOptions {
  /** 日志目录（默认 ./logs） */
  dir?: string;
  /** 日志文件名前缀（默认 fhcode） */
  filename?: string;
  /** 最大文件大小，支持 '10m'/'100k'/'1g'（默认 '10m'） */
  maxSize?: string;
  /** 保留的最大文件数（默认 10） */
  maxFiles?: number;
  /** 保留天数（默认 7 天） */
  maxDays?: number;
  /** 文件扩展名（默认 .log） */
  extension?: string;
}

export class LogRotator {
  private options: Required<LogRotatorOptions>;
  private currentFile: string;
  private currentSize: number = 0;
  private currentDate: string;
  private stream: fs.WriteStream | null = null;

  constructor(options: LogRotatorOptions = {}) {
    this.options = {
      dir: options.dir || './logs',
      filename: options.filename || 'fhcode',
      maxSize: options.maxSize || '10m',
      maxFiles: options.maxFiles || 10,
      maxDays: options.maxDays || 7,
      extension: options.extension || '.log'
    };

    if (!fs.existsSync(this.options.dir)) {
      fs.mkdirSync(this.options.dir, { recursive: true });
    }

    this.currentDate = this.getDateString();
```

```
1501
1502    for (let i = 1; i <= m; i++) {
1503      for (let j = 1; j <= n; j++) {
1504        const cost = a[i - 1] === b[j - 1] ? 0 : 1;
1505        dp[i][j] = Math.min(
1506          dp[i - 1][j] + 1,
1507          dp[i][j - 1] + 1,
1508          dp[i - 1][j - 1] + cost,
1509        );
1510      }
1511    }
1512    return dp[m][n];
1513  }
1514
1515  /**
1516   * 计算编辑相似度（1 - 编辑距离 / 最大长度）
1517   */
1518  function editSimilarity(a: string, b: string): number {
1519    if (a === b) return 1;
1520    const maxLen = Math.max(a.length, b.length);
1521    if (maxLen === 0) return 1;
1522    const dist = levenshtein(a, b);
1523    return 1 - dist / maxLen;
1524  }
1525
1526  /**
1527   * 检查前缀匹配（预测是否以期望的前 N 个字符开头，或反之）
1528   */
1529  function prefixMatchScore(predicted: string, expected: string): number {
1530    if (predicted === expected) return 1;
1531    const minLen = Math.min(predicted.length, expected.length);
1532    if (minLen === 0) return 0;
1533    let match = 0;
1534    for (let i = 0; i < minLen; i++) {
1535      if (predicted[i] === expected[i]) match++;
1536      else break;
1537    }
1538    return match / minLen;
1539  }
1540
1541  /**
1542   * 检查第一个 token 是否匹配（按空白/符号分割）
1543   */
1544  function firstTokenMatch(predicted: string, expected: string): boolean {
1545    const tokenize = (s: string) => s.trim().split(/[\s;,.(){}[\]]+/).filter(Boolean);
1546    const predTokens = tokenize(predicted);
1547    const expTokens = tokenize(expected);
1548    if (predTokens.length === 0 || expTokens.length === 0) return false;
1549    return predTokens[0] === expTokens[0];
1550  }
```

```
1551
1552  /**
1553   * 补全模型评估器
1554   */
1555  export class CompletionEvaluator {
1556    private samples: EvalSample[] = [];
1557
1558    /**
1559     * 从 JSONL 文件加载评估样本
1560     */
1561    loadSamples(filePath: string): number {
1562      if (!existsSync(filePath)) throw new Error(`评估样本文件不存在: ${filePath}`);
1563      const content = readFileSync(filePath, 'utf-8');
1564      const lines = content.split('\n').filter((l) => l.trim());
1565      this.samples = lines.map((line) => {
1566        const obj = JSON.parse(line);
1567        return {
1568          prefix: obj.prefix,
1569          suffix: obj.suffix,
1570          expected: obj.middle,
1571          file_path: obj.file_path,
1572          language: obj.language,
1573        };
1574      });
1575      return this.samples.length;
1576    }
1577
1578    /**
1579     * 设置评估样本
1580     */
1581    setSamples(samples: EvalSample[]): void {
1582      this.samples = samples;
1583    }
1584
1585    /**
1586     * 运行评估
1587     */
1588    async evaluate(completionFn: CompletionFn, maxSamples?: number): Promise<EvalResult> {
1589      const samples = maxSamples ? this.samples.slice(0, maxSamples) : this.samples;
1590      if (samples.length === 0) throw new Error('没有评估样本');
1591
1592      const details: EvalResult['details'] = [];
1593      const perLanguage: EvalResult['perLanguage'] = {};
1594
1595      let exactMatchCount = 0;
1596      let totalEditSimilarity = 0;
1597      let totalPrefixMatch = 0;
1598      let firstTokenMatchCount = 0;
1599      let totalLatency = 0;
1600
```

```
    for (let i = 0; i < samples.length; i++) {
      const sample = samples[i];
      const { text: predicted, latencyMs } = await completionFn(sample.prefix, sample.suffix, sample.language);

      const exact = predicted === sample.expected;
      const editSim = editSimilarity(predicted, sample.expected);
      const prefixMatch = prefixMatchScore(predicted, sample.expected);
      const firstToken = firstTokenMatch(predicted, sample.expected);

      if (exact) exactMatchCount++;
      totalEditSimilarity += editSim;
      totalPrefixMatch += prefixMatch;
      if (firstToken) firstTokenMatchCount++;
      totalLatency += latencyMs;

      details.push({
        expected: sample.expected,
        predicted,
        exactMatch: exact,
        editSimilarity: editSim,
        latencyMs,
        language: sample.language,
      });

      // 按语言统计
      const lang = sample.language || 'unknown';
      if (!perLanguage[lang]) {
        perLanguage[lang] = { exactMatch: 0, editSimilarity: 0, count: 0 };
      }
      perLanguage[lang].count++;
      if (exact) perLanguage[lang].exactMatch++;
      perLanguage[lang].editSimilarity += editSim;
    }

    // 计算百分比
    for (const lang of Object.keys(perLanguage)) {
      const stat = perLanguage[lang];
      stat.exactMatch = stat.exactMatch / stat.count;
      stat.editSimilarity = stat.editSimilarity / stat.count;
    }

    return {
      exactMatch: exactMatchCount / samples.length,
      editSimilarity: totalEditSimilarity / samples.length,
      prefixMatch: totalPrefixMatch / samples.length,
      firstTokenMatch: firstTokenMatchCount / samples.length,
      avgLatencyMs: totalLatency / samples.length,
      totalSamples: samples.length,
      perLanguage,
      details,
```

```
    };
  }

  /**
   * 格式化评估结果为可读字符串
   */
  formatResult(result: EvalResult): string {
    const lines = [
      '=== 补全模型评估结果 ===',
      `样本数: ${result.totalSamples}`,
      `精确匹配率: ${(result.exactMatch * 100).toFixed(2)}%`,
      `编辑相似度: ${(result.editSimilarity * 100).toFixed(2)}%`,
      `前缀匹配率: ${(result.prefixMatch * 100).toFixed(2)}%`,
      `首Token匹配率: ${(result.firstTokenMatch * 100).toFixed(2)}%`,
      `平均延迟: ${result.avgLatencyMs.toFixed(2)}ms`,
      '',
      '--- 按语言统计 ---',
    ];

    for (const [lang, stat] of Object.entries(result.perLanguage)) {
      lines.push(
        `${lang}: ${stat.count} 样本, 精确匹配 ${(stat.exactMatch * 100).toFixed(1)}%, 编辑相似度 ${(stat.editSimilarity * 100).toFixed(1)}%`,
      );
    }

    return lines.join('\n');
  }

  /**
   * 保存评估结果到 JSON 文件
   */
  saveResult(result: EvalResult, outputPath: string): void {
    const outputDir = outputPath.includes('/') ? outputPath.slice(0, outputPath.lastIndexOf('/')) : '.';
    if (!existsSync(outputDir)) mkdirSync(outputDir, { recursive: true });
    writeFileSync(outputPath, JSON.stringify(result, null, 2), 'utf-8');
  }
}

/**
 * 便捷函数：评估补全模型
 */
export async function evaluateCompletion(
  samplesPath: string,
  completionFn: CompletionFn,
  maxSamples?: number,
): Promise<EvalResult> {
  const evaluator = new CompletionEvaluator();
  evaluator.loadSamples(samplesPath);
  return evaluator.evaluate(completionFn, maxSamples);
}
```

```
1701
```

src/training/fim-data.ts

```
1702 /**
1703  * 飞虹 Code - FIM (Fill-In-the-Middle) 训练数据准备 (P3-1)
1704  *
1705  * 从代码库生成 FIM 格式的训练数据，用于专用补全模型微调。
1706  * FIM 格式：将代码随机分成 prefix / suffix / middle 三部分，
1707  * 模型学习根据 prefix 和 suffix 预测 middle（即光标位置的补全）。
1708  */
1709 import { readFileSync, writeFileSync, existsSync, mkdirSync, readdirSync, statSync } from 'fs';
1710 import { join, extname, relative } from 'path';
1711 import { logger } from '../shared/logger';
1712
1713 /** 支持的编程语言及其扩展名 */
1714 const LANGUAGE_EXTENSIONS: Record<string, string[]> = {
1715   typescript: ['.ts', '.tsx'],
1716   javascript: ['.js', '.jsx', '.mjs', '.cjs'],
1717   python: ['.py'],
1718   java: ['.java'],
1719   go: ['.go'],
1720   rust: ['.rs'],
1721   cpp: ['.cpp', '.cc', '.cxx', '.h', '.hpp'],
1722   c: ['.c', '.h'],
1723   csharp: ['.cs'],
1724   php: ['.php'],
1725   ruby: ['.rb'],
1726   swift: ['.swift'],
1727   kotlin: ['.kt', '.kts'],
1728   html: ['.html', '.htm'],
1729   css: ['.css', '.scss', '.less'],
1730   sql: ['.sql'],
1731   shell: ['.sh', '.bash', '.zsh'],
1732   yaml: ['.yml', '.yaml'],
1733   json: ['.json'],
1734   markdown: ['.md', '.markdown'],
1735 };
1736
1737 /** 忽略的目录和文件 */
1738 const IGNORE_PATTERNS = [
1739   'node_modules', '.git', 'dist', 'build', 'out',
1740   '.next', '.nuxt', '.cache', '.turbo',
1741   'coverage', '.nyc_output',
1742   'vendor', 'third_party',
1743   '__pycache__', '.pytest_cache',
1744   'target', 'bin', 'obj',
1745   '.idea', '.vscode',
1746   'package-lock.json', 'yarn.lock', 'pnpm-lock.yaml',
1747 ];
1748
1749 /** FIM 训练样本 */
```

```
export interface FimSample {
  prefix: string;
  suffix: string;
  middle: string;
  file_path: string;
  language: string;
  cursor_offset: number;
  middle_length: number;
}

/** 数据准备配置 */
export interface FimDataConfig {
  sourceDir: string;
  outputPath: string;
  samplesPerFile?: number;
  maxFileSize?: number;
  minMiddleLength?: number;
  maxMiddleLength?: number;
  minPrefixLength?: number;
  minSuffixLength?: number;
  languages?: string[];
  trainRatio?: number;
  seed?: number;
}

const DEFAULT_CONFIG: Required<Omit<FimDataConfig, 'sourceDir' | 'outputPath'>> = {
  samplesPerFile: 5,
  maxFileSize: 100 * 1024,
  minMiddleLength: 10,
  maxMiddleLength: 200,
  minPrefixLength: 50,
  minSuffixLength: 10,
  languages: [],
  trainRatio: 0.9,
  seed: 42,
};

/** 可复现随机数生成器（Mulberry32） */
function createRng(seed: number): () => number {
  let a = seed;
  return function () {
    a |= 0;
    a = (a + 0x6d2b79f5) | 0;
    let t = Math.imul(a ^ (a >>> 15), 1 | a);
    t = (t + Math.imul(t ^ (t >>> 7), 61 | t)) ^ t;
    return ((t ^ (t >>> 14)) >>> 0) / 4294967296;
  };
}

function detectLanguage(filePath: string): string | null {
```

```
  const ext = extname(filePath).toLowerCase();
  for (const [lang, exts] of Object.entries(LANGUAGE_EXTENSIONS)) {
    if (exts.includes(ext)) return lang;
  }
  return null;
}

function shouldIgnore(filePath: string): boolean {
  for (const pattern of IGNORE_PATTERNS) {
    if (filePath.includes(pattern)) return true;
  }
  return false;
}

function scanFiles(dir: string, languages: string[]): Array<{ path: string; language: string }> {
  const results: Array<{ path: string; language: string }> = [];
  function scan(currentDir: string) {
    let entries: string[];
    try { entries = readdirSync(currentDir); } catch { return; }
    for (const entry of entries) {
      const fullPath = join(currentDir, entry);
      if (shouldIgnore(fullPath)) continue;
      let stats;
      try { stats = statSync(fullPath); } catch { continue; }
      if (stats.isDirectory()) {
        scan(fullPath);
      } else if (stats.isFile()) {
        const language = detectLanguage(fullPath);
        if (language && (languages.length === 0 || languages.includes(language))) {
          results.push({ path: fullPath, language });
        }
      }
    }
  }
  scan(dir);
  return results;
}

function generateFimSample(
  content: string,
  rng: () => number,
  config: Required<Omit<FimDataConfig, 'sourceDir' | 'outputPath'>>,
): FimSample | null {
  const totalLength = content.length;
  for (let attempt = 0; attempt < 10; attempt++) {
    const middleLength = Math.floor(
      config.minMiddleLength + rng() * (config.maxMiddleLength - config.minMiddleLength),
    );
    const minCursor = config.minPrefixLength;
    const maxCursor = totalLength - middleLength - config.minSuffixLength;
```

```
    if (maxCursor <= minCursor) continue;
    const cursorOffset = Math.floor(minCursor + rng() * (maxCursor - minCursor));
    const prefix = content.slice(0, cursorOffset);
    const middle = content.slice(cursorOffset, cursorOffset + middleLength);
    const suffix = content.slice(cursorOffset + middleLength);
    if (prefix.length < config.minPrefixLength) continue;
    if (suffix.length < config.minSuffixLength) continue;
    if (middle.length < config.minMiddleLength) continue;
    if (middle.trim().length === 0) continue;
    return {
      prefix, suffix, middle,
      file_path: '', language: '',
      cursor_offset: cursorOffset,
      middle_length: middleLength,
    };
  }
  return null;
}

export class FimDataPreparer {
  private config: Required<FimDataConfig>;
  private rng: () => number;

  constructor(config: FimDataConfig) {
    this.config = { ...DEFAULT_CONFIG, ...config } as Required<FimDataConfig>;
    this.rng = createRng(this.config.seed);
  }

  prepare(): { trainCount: number; valCount: number; totalFiles: number; outputPath: string } {
    const { sourceDir, outputPath, samplesPerFile, maxFileSize, trainRatio } = this.config;
    logger.info('开始准备 FIM 训练数据', { sourceDir, outputPath });

    const files = scanFiles(sourceDir, this.config.languages);
    logger.info(`扫描到 ${files.length} 个代码文件`);
    if (files.length === 0) throw new Error(`未在 ${sourceDir} 中找到任何代码文件`);

    const allSamples: FimSample[] = [];
    for (const { path: filePath, language } of files) {
      let stats;
      try { stats = statSync(filePath); } catch { continue; }
      if (stats.size > maxFileSize) continue;
      let content: string;
      try { content = readFileSync(filePath, 'utf-8'); } catch { continue; }
      if (content.length < this.config.minPrefixLength + this.config.minMiddleLength + this.config.minSuffixLength) continue;

      const relPath = relative(sourceDir, filePath);
      for (let i = 0; i < samplesPerFile; i++) {
        const sample = generateFimSample(content, this.rng, this.config);
        if (sample) {
          sample.file_path = relPath;
```

```
          sample.language = language;
          allSamples.push(sample);
        }
      }
    }

    logger.info(`生成 ${allSamples.length} 个 FIM 样本`);
    if (allSamples.length === 0) throw new Error('未能生成任何有效样本');

    // 打乱
    for (let i = allSamples.length - 1; i > 0; i--) {
      const j = Math.floor(this.rng() * (i + 1));
      [allSamples[i], allSamples[j]] = [allSamples[j], allSamples[i]];
    }

    const trainCount = Math.floor(allSamples.length * trainRatio);
    const trainSamples = allSamples.slice(0, trainCount);
    const valSamples = allSamples.slice(trainCount);

    const outputDir = outputPath.includes('/') ? outputPath.slice(0, outputPath.lastIndexOf('/')) : '.';
    if (outputDir && !existsSync(outputDir)) mkdirSync(outputDir, { recursive: true });

    const trainPath = outputPath.replace('.jsonl', '.train.jsonl');
    writeFileSync(trainPath, trainSamples.map((s) => JSON.stringify(s)).join('\n'), 'utf-8');

    const valPath = outputPath.replace('.jsonl', '.val.jsonl');
    writeFileSync(valPath, valSamples.map((s) => JSON.stringify(s)).join('\n'), 'utf-8');

    const metadata = {
      source_dir: sourceDir,
      total_files: files.length,
      total_samples: allSamples.length,
      train_samples: trainSamples.length,
      val_samples: valSamples.length,
      train_ratio: trainRatio,
      config: {
        samples_per_file: samplesPerFile,
        max_file_size: maxFileSize,
        min_middle_length: this.config.minMiddleLength,
        max_middle_length: this.config.maxMiddleLength,
        min_prefix_length: this.config.minPrefixLength,
        min_suffix_length: this.config.minSuffixLength,
        languages: this.config.languages,
        seed: this.config.seed,
      },
      language_distribution: this.countByLanguage(allSamples),
      created_at: new Date().toISOString(),
    };

    const metaPath = outputPath.replace('.jsonl', '.meta.json');
```

```
    writeFileSync(metaPath, JSON.stringify(metadata, null, 2), 'utf-8');

    logger.info('FIM 训练数据准备完成', { train: trainSamples.length, val: valSamples.length });
    return { trainCount: trainSamples.length, valCount: valSamples.length, totalFiles: files.length, outputPath };
  }

  private countByLanguage(samples: FimSample[]): Record<string, number> {
    const counts: Record<string, number> = {};
    for (const s of samples) counts[s.language] = (counts[s.language] || 0) + 1;
    return counts;
  }
}

export function prepareFimData(config: FimDataConfig) {
  return new FimDataPreparer(config).prepare();
}
```

┌── src/voice/voice-programming.ts ──┐

```
/**
 * 飞虹 Code - 语音编程 (阶段四-1)
 *
 * 支持语音编程能力：
 * - 语音指令解析：将自然语言语音转换为编程操作
 * - 语音转代码：根据语音描述生成代码
 * - 语音命令：常用操作的语音快捷命令
 * - 语音上下文：维护语音对话上下文
 */
import { logger } from '../shared/logger';

/** 语音指令类型 */
export type VoiceCommandType =
  | 'new_file' | 'open_file' | 'save_file' | 'close_file'
  | 'run_code' | 'debug_code' | 'stop_code'
  | 'search' | 'replace' | 'goto_line'
  | 'comment' | 'uncomment' | 'format'
  | 'undo' | 'redo' | 'copy' | 'paste' | 'cut'
  | 'zoom_in' | 'zoom_out' | 'reset_zoom'
  | 'toggle_terminal' | 'toggle_sidebar' | 'toggle_fullscreen'
  | 'generate_code' | 'explain_code' | 'refactor_code' | 'review_code'
  | 'chat' | 'unknown';

/** 语音指令 */
export interface VoiceCommand {
  type: VoiceCommandType;
  rawText: string;
  params: Record<string, any>;
  confidence: number;
  needConfirm: boolean;
}
```

```
/** 语音转代码结果 */
export interface VoiceToCodeResult {
  code: string;
  language: string;
  explanation: string;
  confidence: number;
}

/** 语音对话上下文 */
export interface VoiceContext {
  sessionId: string;
  history: Array<{
    role: 'user' | 'assistant';
    text: string;
    timestamp: string;
  }>;
  currentFile?: string;
  currentLanguage?: string;
  createdAt: string;
  lastActiveAt: string;
}

/** 语音命令映射规则 */
const COMMAND_RULES: Array<{
  type: VoiceCommandType;
  patterns: RegExp[];
  extractParams?: (text: string) => Record<string, any>;
  needConfirm?: boolean;
}> = [
  {
    type: 'new_file',
    patterns: [/新建文件|创建文件|新文件|new file|create file|创建一个?/i],
    extractParams: (text) => {
      const match = text.match(/(?:新建|创建|new|create)(?:文件|file)?(?:一个)?(?:叫|名为|named)?\s*([\w.-]
+)/i);
      return match ? { fileName: match[1] } : {};
    },
  },
  // 面板/视图类开关（语义更具体）须排在 open_file 之前：
  // 「打开终端/侧边栏/全屏」与 open_file 的宽泛「打开」匹配置信度相同，
  // 并列时先匹配者胜，若排在后面会被 open_file 错误抢占。
  { type: 'toggle_terminal', patterns: [/终端|terminal|控制台/i] },
  { type: 'toggle_sidebar', patterns: [/侧边栏|侧栏|sidebar/i] },
  { type: 'toggle_fullscreen', patterns: [/全屏|fullscreen/i] },
  {
    type: 'open_file',
    patterns: [/打开文件|打开|open file|open/i],
    extractParams: (text) => {
      const match = text.match(/(?:打开|open)\s*(?:文件|file)?\s*([\w./]+)/i);
      return match ? { fileName: match[1] } : {};
    },
```

```
  },
  { type: 'save_file', patterns: [/保存|存盘|save/i] },
  { type: 'close_file', patterns: [/关闭文件|关掉文件|close file|close/i] },
  { type: 'run_code', patterns: [/运行|执行|跑一下|run|execute/i] },
  { type: 'debug_code', patterns: [/调试|debug/i] },
  { type: 'stop_code', patterns: [/停止|终止|stop/i] },
  {
    type: 'search',
    patterns: [/搜索|查找|search|find/i],
    extractParams: (text) => {
      const match = text.match(/(?:搜索|查找|search|find)\s*(?:什么|什么是|for)?\s*(.+)/i);
      return match ? { query: match[1].trim() } : {};
    },
  },
  {
    type: 'replace',
    patterns: [/替换|replace/i],
    extractParams: (text) => {
      const m = text.match(/(?:把|将)?\s*([^\s，。]+?)\s*(?:替换|replace)\s*(?:成|为|换成)?\s*([^\s，。]+)/
i);
      return m ? { from: m[1], to: m[2] } : {};
    },
  },
  {
    type: 'goto_line',
    patterns: [/跳到第?\d+行|跳转到|goto line|go to line/i],
    extractParams: (text) => {
      const match = text.match(/(\d+)\s*行/);
      return match ? { line: parseInt(match[1], 10) } : {};
    },
  },
  { type: 'comment', patterns: [/注释|comment/i] },
  { type: 'uncomment', patterns: [/取消注释|去掉注释|uncomment/i] },
  { type: 'format', patterns: [/格式化|format|美化代码/i] },
  { type: 'undo', patterns: [/撤销|undo/i] },
  { type: 'redo', patterns: [/重做|恢复|redo/i] },
  { type: 'copy', patterns: [/复制|拷贝|copy/i] },
  { type: 'paste', patterns: [/粘贴|paste/i] },
  { type: 'cut', patterns: [/剪切|cut/i] },
  {
    type: 'generate_code',
    patterns: [/生成代码|写代码|帮我写|generate code|write code/i],
    extractParams: (text) => {
      const match = text.match(/(?:生成|写|帮我写|generate|write)\s*(?:代码|code)?\s*(?:实现|做|一个|一个)?\s
*(.+)/i);
      return match ? { description: match[1].trim() } : {};
    },
    needConfirm: true,
  },
  { type: 'explain_code', patterns: [/解释|讲解|explain/i] },
  { type: 'refactor_code', patterns: [/重构|优化代码|refactor/i], needConfirm: true },
  { type: 'review_code', patterns: [/审查代码|代码审查|review code|code review/i] },
```

```
  { type: 'zoom_in', patterns: [/放大|zoom in/i] },
  { type: 'zoom_out', patterns: [/缩小|zoom out/i] },
  { type: 'reset_zoom', patterns: [/重置缩放|reset zoom/i] },
];

/**
 * 语音编程管理器
 */
export class VoiceProgrammingManager {
  private contexts: Map<string, VoiceContext> = new Map();

  constructor() {
    logger.info('voice programming manager initialized');
  }

  /**
   * 解析语音指令
   */
  parseCommand(text: string): VoiceCommand {
    const trimmed = text.trim();
    let bestMatch: { type: VoiceCommandType; confidence: number; params: Record<string, any>; needConfirm: boolean } | null = null;

    for (const rule of COMMAND_RULES) {
      for (const pattern of rule.patterns) {
        if (pattern.test(trimmed)) {
          const match = trimmed.match(pattern);
          const matchLength = match ? match[0].length : 0;
          const confidence = Math.min(1, matchLength / trimmed.length + 0.3);
          const params = rule.extractParams ? rule.extractParams(trimmed) : {};

          if (!bestMatch || confidence > bestMatch.confidence) {
            bestMatch = { type: rule.type, confidence, params, needConfirm: rule.needConfirm || false };
          }
        }
      }
    }

    if (!bestMatch) {
      return { type: 'chat', rawText: trimmed, params: { text: trimmed }, confidence: 0.5, needConfirm: false };
    }

    return {
      type: bestMatch.type,
      rawText: trimmed,
      params: bestMatch.params,
      confidence: bestMatch.confidence,
      needConfirm: bestMatch.needConfirm,
    };
  }

```

```
  /**
   * 语音转代码
   */
  async voiceToCode(description: string, language: string = 'typescript', _context?: string): Promise<VoiceToCodeResult> {
    logger.info('voice to code', { description, language });
    const code = this.generateCodeFromTemplate(description, language);
    return {
      code,
      language,
      explanation: `根据语音描述"${description}"生成的${language}代码`,
      confidence: 0.7,
    };
  }

  /**
   * 基于模板生成代码
   */
  private generateCodeFromTemplate(description: string, language: string): string {
    const desc = description.toLowerCase();

    if (desc.includes('函数') || desc.includes('function')) {
      const nameMatch = description.match(/(?:函数|function)\s*(?:叫|名为)?\s*(\w+)/);
      const name = nameMatch ? nameMatch[1] : 'myFunction';
      if (language === 'typescript' || language === 'javascript') {
        return `function ${name}(params: any): any {\n  // TODO: 实现逻辑\n  return null;\n}`;
      }
    }

    if (desc.includes('类') || desc.includes('class')) {
      const nameMatch = description.match(/(?:类|class)\s*(?:叫|名为)?\s*(\w+)/);
      const name = nameMatch ? nameMatch[1] : 'MyClass';
      if (language === 'typescript') {
        return `class ${name} {\n  constructor() {\n    // TODO: 初始化\n  }\n\n  method(): void {\n    // TODO: 实现方法\n  }\n}`;
      }
    }

    return `// ${description}\n// TODO: 实现以下功能\n`;
  }

  /**
   * 创建语音会话上下文
   */
  createContext(sessionId?: string): VoiceContext {
    const id = sessionId || `voice-${Date.now()}-${Math.random().toString(36).slice(2)}`;
    const context: VoiceContext = {
      sessionId: id,
      history: [],
      createdAt: new Date().toISOString(),
      lastActiveAt: new Date().toISOString(),
    };
```

```
    this.contexts.set(id, context);
    return context;
  }

  getContext(sessionId: string): VoiceContext | undefined {
    return this.contexts.get(sessionId);
  }

  addHistory(sessionId: string, role: 'user' | 'assistant', text: string): void {
    const context = this.contexts.get(sessionId);
    if (!context) return;
    context.history.push({ role, text, timestamp: new Date().toISOString() });
    context.lastActiveAt = new Date().toISOString();
    if (context.history.length > 50) context.history = context.history.slice(-50);
  }

  setCurrentFile(sessionId: string, filePath: string, language?: string): void {
    const context = this.contexts.get(sessionId);
    if (!context) return;
    context.currentFile = filePath;
    if (language) context.currentLanguage = language;
    context.lastActiveAt = new Date().toISOString();
  }

  cleanupExpiredSessions(maxAgeMs: number = 30 * 60 * 1000): number {
    const now = Date.now();
    let cleaned = 0;
    for (const [id, context] of this.contexts) {
      if (now - new Date(context.lastActiveAt).getTime() > maxAgeMs) {
        this.contexts.delete(id);
        cleaned++;
      }
    }
    if (cleaned > 0) logger.info('voice sessions cleaned', { cleaned });
    return cleaned;
  }

  getSupportedCommands(): Array<{ type: VoiceCommandType; description: string; examples: string[] }> {
    return [
      { type: 'new_file', description: '新建文件', examples: ['新建文件', '创建一个叫 utils 的文件'] },
      { type: 'open_file', description: '打开文件', examples: ['打开 index.ts', '打开配置文件'] },
      { type: 'save_file', description: '保存文件', examples: ['保存', '存盘'] },
      { type: 'close_file', description: '关闭文件', examples: ['关闭文件', 'close file'] },
      { type: 'run_code', description: '运行代码', examples: ['运行', '执行一下', '跑一下'] },
      { type: 'debug_code', description: '调试代码', examples: ['调试', 'debug'] },
      { type: 'stop_code', description: '停止代码', examples: ['停止', '终止运行'] },
      { type: 'search', description: '搜索', examples: ['搜索函数', '查找 TODO'] },
      { type: 'replace', description: '替换', examples: ['把 A 替换成 B'] },
      { type: 'goto_line', description: '跳转到指定行', examples: ['跳到第100行', '跳转到50行'] },
      { type: 'comment', description: '注释代码', examples: ['注释', '注释这行'] },
```

```
      { type: 'uncomment', description: '取消注释', examples: ['取消注释', 'uncomment'] },
      { type: 'format', description: '格式化代码', examples: ['格式化', '美化代码'] },
      { type: 'undo', description: '撤销', examples: ['撤销', 'undo'] },
      { type: 'redo', description: '重做', examples: ['重做', '恢复', 'redo'] },
      { type: 'copy', description: '复制', examples: ['复制', 'copy'] },
      { type: 'paste', description: '粘贴', examples: ['粘贴', 'paste'] },
      { type: 'cut', description: '剪切', examples: ['剪切', 'cut'] },
      { type: 'generate_code', description: '生成代码', examples: ['生成一个排序函数', '帮我写一个登录接口'] },
      { type: 'explain_code', description: '解释代码', examples: ['解释这段代码', 'explain'] },
      { type: 'refactor_code', description: '重构代码', examples: ['重构', '优化这段代码'] },
      { type: 'review_code', description: '审查代码', examples: ['审查代码', 'code review'] },
      { type: 'toggle_terminal', description: '切换终端', examples: ['打开终端', '关闭终端'] },
      { type: 'toggle_sidebar', description: '切换侧边栏', examples: ['打开侧边栏', '关闭侧栏'] },
      { type: 'toggle_fullscreen', description: '切换全屏', examples: ['全屏', '退出全屏'] },
      { type: 'zoom_in', description: '放大', examples: ['放大', 'zoom in'] },
      { type: 'zoom_out', description: '缩小', examples: ['缩小', 'zoom out'] },
      { type: 'reset_zoom', description: '重置缩放', examples: ['重置缩放', 'reset zoom'] },
    ];
  }
}

export function createVoiceProgrammingManager(): VoiceProgrammingManager {
  return new VoiceProgrammingManager();
}

```

┌── src/shared/concurrency.ts ──┐

```
/**
 * 飞虹 Code (对标 Muse Code · 自研内核)
 * 晋江市飞虹智科技企业管理有限公司 · 飞扬企源研发中心 · 负责人：吴赐虹
 *
 * 并发控制工具：限制异步任务的最大并发数，防止 API 限流和资源耗尽。
 */

/**
 * 异步任务池：限制最大并发数。
 * 类似 p-limit，但零依赖，适用于需要控制模型调用、文件IO等并发场景。
 *
 * @example
 * const limit = asyncPool(3);
 * const results = await Promise.all(tasks.map(t => limit(() => fetch(t))));
 */
export function asyncPool(concurrency: number) {
  const max = Math.max(1, concurrency);
  let active = 0;
  const queue: Array<() => void> = [];

  const next = () => {
    if (active < max && queue.length > 0) {
      const resolve = queue.shift()!;
      active++;
```

```
      resolve();
    }
  };

  const acquire = (): Promise<void> => {
    if (active < max) {
      active++;
      return Promise.resolve();
    }
    return new Promise<void>((resolve) => queue.push(resolve));
  };

  const release = () => {
    active--;
    next();
  };

  return <T>(fn: () => Promise<T>): Promise<T> =>
    acquire().then(() =>
      fn().then(
        (result) => {
          release();
          return result;
        },
        (error) => {
          release();
          throw error;
        },
      ),
    );
}

/**
 * 批量执行异步任务，限制最大并发数。
 * 自动保留原始顺序，所有任务完成后返回结果数组。
 *
 * @param tasks 任务工厂函数数组
 * @param concurrency 最大并发数（默认 3）
 * @returns 按原始顺序排列的结果数组
 *
 * @example
 * const results = await runWithConcurrency(
 *   urls.map(url => () => fetch(url)),
 *   3
 * );
 */
export async function runWithConcurrency<T>(
  tasks: Array<() => Promise<T>>,
  concurrency: number = 3,
): Promise<T[]> {
```

```
  const limit = asyncPool(concurrency);
  return Promise.all(tasks.map((task) => limit(task)));
}

/**
 * 批量执行异步任务，限制最大并发数，允许部分失败。
 * 类似 Promise.allSettled，但有并发限制。
 *
 * @param tasks 任务工厂函数数组
 * @param concurrency 最大并发数（默认 3）
 * @returns PromiseSettledResult 数组
 */
export async function runWithConcurrencySettled<T>(
  tasks: Array<() => Promise<T>>,
  concurrency: number = 3,
): Promise<Array<PromiseSettledResult<T>>> {
  const limit = asyncPool(concurrency);
  return Promise.allSettled(tasks.map((task) => limit(task)));
}

┌── src/shared/i18n.ts ──┐
/**
 * 飞虹 Code (对标 Muse Code · 自研内核)
 * 晋江市飞虹智科技企业管理有限公司 · 飞扬企源研发中心 · 负责人：吴赐虹
 *
 * 国际化（i18n）基础设施：中英文双语界面支持。
 *  - t(key, params?)        取当前语言字符串并做 {name} 插值
 *  - detectLang()           自动检测语言（FHCODE_LANG > 系统 locale）
 *  - setLang/getLang        运行时切换（供 --lang 参数使用）
 *
 * 设计：纯数据 + 轻量函数，无外部依赖，避免循环引用。
 */
export type Lang = 'zh' | 'en';

/** 归一化任意语言写法为 'zh' | 'en' | null（无法识别返回 null） */
export function normalizeLang(raw?: string | null): Lang | null {
  if (!raw) return null;
  const v = raw.trim().toLowerCase();
  if (v === 'zh' || v === 'zh-cn' || v === 'zh-tw' || v === 'chinese' || v === '中文') return 'zh';
  if (v === 'en' || v === 'en-us' || v === 'en-gb' || v === 'english') return 'en';
  if (v.startsWith('zh')) return 'zh';
  if (v.startsWith('en')) return 'en';
  return null;
}

/** 自动检测当前语言：FHCODE_LANG 优先，其次系统 locale，未识别默认中文 */
export function detectLang(): Lang {
  const env = normalizeLang(process.env.FHCODE_LANG);
  if (env) return env;
  const locale = (
```

```
    process.env.LANG ||
    process.env.LC_ALL ||
    process.env.LANGUAGE ||
    ''
  ).toLowerCase();
  if (locale.includes('zh') || locale.includes('cn')) return 'zh';
  if (locale.includes('en')) return 'en';
  return 'zh';
}

let currentLang: Lang = detectLang();

export function getLang(): Lang {
  return currentLang;
}

/** 设置当前语言；仅接受 'zh' / 'en'，其它忽略（保持现状） */
export function setLang(lang?: Lang | string | null): void {
  const norm = normalizeLang(lang);
  if (norm) currentLang = norm;
}

/* ============================ 中文词条 ============================ */
const ZH: Record<string, string> = {
  'app.product': '飞虹 Code',
  'app.tagline': '终端 AI 编程智能体（对标 Muse Code / Claude Code / Cursor CLI · 自研内核差异化）',
  'app.signature':
    '晋江市飞虹智科技企业管理有限公司 · 飞扬企源研发中心 · 负责人：吴赐虹',
  'cli.brand': '[飞虹 Code]',
  'cli.help': `飞虹 Code (fhcode) v{version}

用法:
  fhcode                              进入交互 REPL
  fhcode "<需求>"                     单命令模式执行一条需求
  fhcode --parallel "<需求>"          多子代理并行（M2: git worktree 隔离）
  fhcode /plan "<目标>"               生成实现计划（只读）
  fhcode /grill [路径]                红队式代码审查（只读）
  fhcode /goal "<目标>"               分解并保存高层目标
  fhcode sessions                     列出历史会话（M3 恢复与审计）
  fhcode resume <id>                  从检查点恢复中断的会话（M3）
  fhcode diff [<id>]                  展示会话/工作区变更（M3）
  fhcode rollback <id> [--yes]        回滚会话改动（M3，危险操作需 --yes）

企业能力 (M4):
  fhcode whoami                       当前租户 / 用户 / 角色 / 隔离目录 / 配额
  fhcode policy                       查看生效的 RBAC 策略与角色矩阵
  fhcode audit [--limit N]            查看审计记录（默认最近 20 条）
  fhcode audit verify                 校验审计哈希链是否被篡改
  fhcode tenants                      列出全部租户与用量汇总
  fhcode doctor                       环境自检（版本/git/provider/路径/网络）
```

```
2451
2452  自我进化 (M6):
2453    fhcode model-stats                  查看各模型性能统计
2454    fhcode experiences [路径]           列出经验库
2455
2456  Web 控制台 (M5):
2457    fhcode serve [--port 8080]          启动 Web 管理控制台（默认 http://localhost:8080）
2458                                        令牌由 FH_WEB_TOKEN 或自动生成（控制台仅观测，不执行）
2459
2460  自主编程 (M8):
2461    fhcode code-write "<目标>"          自主编写代码（规划→编写→测试→审查→修复）
2462    fhcode quality-gate [路径]          质量门禁审查（安全+质量+测试覆盖）
2463    fhcode self-improve                 自我改进统计与历史
2464
2465  全自动软件工程 Agent (M9):
2466    fhcode swe "<目标>"                 读取仓库→拆解规划→实现+验证+自愈→报告
2467                                        --repo <路径> 指定仓库(默认当前目录)
2468                                        --plan-only 仅规划不执行
2469                                        --verify-only 仅跑验证不实现
2470                                        --max-tasks N 限制子任务数(默认8)
2471                                        --max-retries N 单任务自愈重试(默认2)
2472
2473    fhcode --version                    显示版本 (-v)
2474    fhcode --help                       显示帮助 (-h)
2475    fhcode --lang <zh|en>               设置界面语言（中文/英文）
2476
2477  说明: 未配置 FH_PROVIDERS 时自动进入离线模式（脚本化 Mock 驱动闭环验证）。
2478  配置 FH_PROVIDERS（OpenAI 兼容 / Ollama）后，将调用真实大模型执行任务。
2479  企业模式默认开启（租户隔离 + RBAC + 审计链 + 配额），可用 FH_ENTERPRISE=false 关闭。
2480  署名: {signature}`,
2481
2482    'repl.welcome': '飞虹 Code REPL（输入需求回车执行，exit 退出）',
2483    'repl.hint': '提示：未配置 FH_PROVIDERS 时自动离线模式运行；支持 /plan /grill /goal 技能。\n',
2484    'repl.prompt': '飞虹> ',
2485    'repl.errorPrefix': '执行出错: ',
2486    'repl.unknownSkill': '未知技能: /{cmd}（支持 /plan /grill /goal）',
2487    'repl.bye': '\n再见。',
2488    'repl.stateReady': '就绪',
2489    'repl.stateRunning': '运行中',
2490
2491    'stream.toolCalling': '调用工具: {tools}',
2492    'stream.toolOk': '成功',
2493    'stream.toolFail': '失败',
2494    'stream.selfHeal': '自愈重试: {category}',
2495    'stream.compact': '上下文压缩: {from} → {to} 条消息',
2496    'stream.done': '任务结束 · 迭代 {iter} 次 · 成本 {cost}',
2497
2498    'run.identity': '身份 tenant={tenant} user={user} role={role}',
2499    'run.start': '开始任务 (runId={id}{mode})',
2500    'run.resultTitle': '===== 执行结果 =====',
```

```
  'run.summary': '迭代 {iter} 次 · 成本 {cost} · 日志 {log}',
  'run.checkpoint':
    '会话检查点: {path}（可用 fhcode sessions / resume / diff 管理）',
  'run.offlineFileYes': '(离线模式演示文件已写入: {file})',
  'run.offlineFileNo':
    '(离线模式未产生演示文件，工作区: {cwd}；如为策略拒绝可用 fhcode audit 查看原因)',
  'run.offlineFragment': ', 离线模式',
  'run.modeOffline': '离线',
  'run.modeLive': '真实',

  'run.parallelMode': '[飞虹 Code] 并行模式: {offline}',
  'run.parallelResult': '===== 并行执行结果 =====',
  'run.parallelRepo': '仓库根: {root} · 工作树已清理: {trees}',

  'run.noSessions': '（无历史会话）',
  'run.sessionList': '历史会话（{mode}模式，目录 {home}）:',
  'run.sessionItem':
    '- {id} | {status} | 迭代{iter} | 成本{cost} | 文件{files} | {time}',
  'run.sessionGoal': '    目标: {goal}',

  'run.resumeDone': '会话 {id} 已完成，无需恢复。',
  'run.resumeStart': '[飞虹 Code] 恢复会话 {id} (状态: {status}, 已迭代 {iter} 次)',

  'run.diffSession': '[飞虹 Code] 会话 {id} 的变更 (cwd={cwd}):',
  'run.diffCwd': '[飞虹 Code] 当前目录 ({cwd}) 工作区变更:',

  'run.rollbackStart': '[飞虹 Code] 回滚会话 {id} 的 {n} 个文件 (cwd={cwd})',
  'run.rollbackReverted': '已还原(已跟踪): {files}',
  'run.rollbackRemoved': '已删除(未跟踪): {files}',
  'run.rollbackNote': '注意: {errors}',

  'err.enterpriseDisabled':
    '企业模式已关闭（FH_ENTERPRISE=false），该命令不可用。',
  'err.runFailed': '[飞虹 Code] 运行失败 ({code}): {message}',
  'err.unexpected': '运行出错: ',
  'err.hint': '（详细日志见结构化 JSON 输出；配置类错误请检查 FH_PROVIDERS / .env）',

  'mode.offline': '离线',
  'mode.live': '真实',

  'audit.empty': '（租户 {tenant} 暂无审计记录，目录 {dir}）',
  'audit.header': '审计记录 {rows}/{all} 条（租户 {tenant}）:',
  'audit.row':
    '#{seq} {ts} [{decision}] {action} by {user}({role}) run={run}',
  'audit.resource': '      资源: {resource}',
  'audit.reason': '      理由: {reason}',
  'audit.chainTail': '链尾哈希: {hash}…',
  'audit.verifyOk': '✅ 审计链完整: {total} 条记录，哈希链自洽未被篡改。',
  'audit.verifyFail': '❌ 审计链校验失败: 共 {total} 条，断点在第 {brokenAt} 条',
```

```
  'tenants.empty': '（暂无租户数据，执行一次任务后自动创建）',
  'tenants.header': '租户用量汇总:',
  'tenants.tableHeader': '  租户ID                会话数    累计成本       审计条数    最近活跃',
  'tenants.row':
    '  {id} {sessions}   {cost}   {audit}   {last}',

  'modelStats.empty': '（暂无模型性能数据，执行任务后自动生成）',
  'modelStats.noRecords': '（无模型统计记录）',
  'modelStats.title': '模型性能统计（M6）:',
  'modelStats.tableHeader':
    '  提供者ID          模型                总调用   成功   失败   成功率   平均延迟   总成本',

  'exp.empty': '（暂无经验记录，完成任务后自动积累）',
  'exp.header': '经验库 ({n} 条，来源: {dir}):',
  'exp.tableHeader': '  ID                                类型                  标题                          成功率  使用次数',
  'exp.more': '  ... 共 {n} 条，显示前 10 条',

  'doctor.title': '===== fhcode doctor 环境自检 =====',
  'doctor.node': 'Node 版本',
  'doctor.git': 'git 可用性',
  'doctor.gitMissing': 'git 不可用（diff/rollback/并行模式将受限）',
  'doctor.config': '模型配置',
  'doctor.configEmpty': '未配置 provider（将自动进入离线模式）',
  'doctor.provider': '供应商',
  'doctor.network': '网络连通',
  'doctor.networkOffline': '离线模式，跳过网络探测',
  'doctor.home': '主目录可写',
  'doctor.sandbox': '沙箱模式',
  'doctor.sandboxUnavailable': '（配置不可用）',
  'doctor.docker': 'Docker 沙箱（container 档）',
  'doctor.allOk': '✅ 环境就绪，无异常项',
  'doctor.issues': '⚠️ 发现 {n} 项异常，请根据提示处理',
  'skillMarket.localSeed': '（网络不可达，已使用本地种子源 templates/market/index.json）',
  'skillMarket.registered': '✅ 已自动注册到本地技能索引（当前共 {n} 个技能）',
  'skillMarket.notRegisteredWarn': '⚠️ 技能已安装但未被本地索引发现：{name}（请检查 SKILL.md 格式）',

  'plugin.installUsage': '用法: fhcode plugin install <本地目录|git URL>',
  'plugin.installed': '✅ 插件已安装: {name}（{dir}）',
  'plugin.installFailed': '❌ 插件安装失败: ',
  'plugin.empty': '（未安装任何插件）',
  'plugin.listTitle': '已安装插件:',

  'skillMarket.localEmpty': '（本地无已安装技能）',
  'skillMarket.localTitle': '本地已安装技能（{n} 个）:',
  'skillMarket.fetchFailed': '❌ 拉取市场索引失败（{base}）: ',
  'skillMarket.schemaWarn': '⚠️ 市场索引 schema 未识别（{schema}），按 0.1.0 兼容处理',
  'skillMarket.searchEmpty': '（市场无匹配 "{q}" 的技能）',
  'skillMarket.searchTitle': '市场「{q}」搜索结果（{n} 个，来源 {base}）:',
  'skillMarket.installHint': '安装: fhcode skill-market install <技能名> [--repo <市场源>]',
  'skillMarket.installUsage': '用法: fhcode skill-market install <技能名> [--repo <市场源>]',
```

```
  'skillMarket.notFound': '❌ 市场中未找到技能: {name}',
  'skillMarket.installed': '✅ 技能已安装: {name}（{dir}），任务中自动发现',
  'skillMarket.installFailed': '❌ 技能安装失败: ',

  'team.start': 'Agent Team 启动（{mode}）: 共享任务清单 + 消息总线协作',
  'team.reportTitle': '===== Agent Team 协作报告 =====',

  'serve.url': '[飞虹 Code] Web 控制台: {url}',
  'serve.token': '[飞虹 Code] 访问令牌 (FH_WEB_TOKEN): {token}',
  'serve.stop': '[飞虹 Code] 按 Ctrl+C 停止',
  'serve.tokenAuto':
    '[飞虹 Code] Web 控制台令牌已自动生成（FH_WEB_TOKEN）: {token}',
  'serve.started': '[飞虹 Code] Web 控制台已启动: http://localhost:{port}',

  'codewrite.resultTitle': '===== M8 自主编写结果 =====',
  'codewrite.files': '生成文件: {files}',

  'quality.failed': '⚠️ {n} 个文件未通过门禁，请修复后再提交',

  'selfimp.title': '===== M6/M8 自我迭代系统状态 =====',
  'selfimp.reflections': '反思次数: {n}',
  'selfimp.successRate': '成功率: {p}%',
  'selfimp.avgDuration': '平均耗时: {ms}ms',
  'selfimp.expLib': '经验库: {n} 条独特经验, 累计验证权重 {w}',
  'selfimp.topExp': 'Top 经验（按被复用次数）:',
  'selfimp.expItem': '  [{count}×] {type} | {title}',
  'selfimp.recent': '最近改进记录:',
  'selfimp.record': '  {ts} | {ok} | 模式: {n} 条',
  'selfimp.improvement': '    → {imp}',
  'selfimp.noRecords': '（暂无改进记录，完成任务后自动生成）',
  'selfimp.learnPreview': '----- 学习提示预览（目标: {goal}）-----',
  'selfimp.noLearned': '（暂无可注入经验）',

  'swe.noProvider':
    '[飞虹 Code] 未配置真实模型供应商，无法进入真实模式。请通过以下任一方式接入: \n' +
    '  1) 环境变量: FH_MODEL_NAME=<模型> FH_MODEL_TYPE=ollama|openai-compatible FH_MODEL_BASE_URL=<地址> [FH_MODEL_API_KEY=<令牌>]\n' +
    '  2) 配置文件: ./fhcode.config.json 的 models.providers 数组\n' +
    '  3) FH_PROVIDERS 环境变量(JSON 数组)\n' +
    '本地 Ollama 示例: FH_MODEL_NAME=qwen2.5-coder:1.5b FH_MODEL_TYPE=ollama fhcode swe "..."',
  'swe.start': '[飞虹 Code] 启动全自动软件工程 Agent（{offline}，仓库={cwd}）',
  'swe.reportTitle': '===== 全自动软件工程 Agent 报告 =====',
  'harness.start': '[飞虹 Code] 启动评测 harness（{mode} 模式 · split={split} · limit={limit}）',
  'harness.noProvider': '[飞虹 Code] 未配置真实模型供应商（请设置 FH_PROVIDERS 或 FH_MODEL_NAME 后使用 --mode real）',
  'harness.reportWritten': '报告已写入: {path}',
  'harness.summary': '汇总: {completed}/{total} 通过（通过率 {rate}%）',

  'approve.prompt': '[审批] 是否允许执行: {action}\n  输入 y/yes 允许，其他拒绝: ',

  'enterprise.whoamiTitle': '当前身份（企业模式）',
  'enterprise.tenantId': '租户 ID',
```

```
  'enterprise.userId': '用户 ID',
  'enterprise.role': '角色',
  'enterprise.sessionDir': '隔离目录',
  'enterprise.quota': '配额',
  'enterprise.quotaUsed': '已用',
  'enterprise.quotaLimit': '上限',
  'enterprise.policyTitle': '生效策略与角色矩阵',
  'enterprise.roleColumn': '角色',
  'enterprise.permissionsColumn': '权限',
  'enterprise.unlimited': '无限',
  'enterprise.unknown': '未知',
  'enterprise.on': '开启',
  'enterprise.off': '关闭 (FH_ENTERPRISE=false)',
  'enterprise.none': '（无）',
  'enterprise.whoami.tenant': '租户 (tenant): {v}',
  'enterprise.whoami.user': '用户 (user)  : {v}',
  'enterprise.whoami.role': '角色 (role)  : {v}',
  'enterprise.whoami.isoDir': '隔离目录     : {v}',
  'enterprise.whoami.session': '  会话       : {v}',
  'enterprise.whoami.audit': '  审计       : {v}',
  'enterprise.whoami.goal': '  目标       : {v}',
  'enterprise.whoami.taskCap': '单任务上限   : {v}',
  'enterprise.whoami.used': '今日已用     : {v}',
  'enterprise.whoami.auditCount': '审计记录数   : {v}',
  'enterprise.whoami.mode': '企业模式     : {v}',
  'policy.version': '策略版本: v{v}',
  'policy.role': '当前角色: {v}',
  'policy.allow': '  直接允许: {v}',
  'policy.approval': '  需审批  : {v}',
  'policy.taskCap': '单任务成本上限: {v}',
  'policy.tenantCap': '租户日成本上限: {v}',
  'policy.denyShell': '危险命令黑名单({n}): {v}',
  'policy.denyPaths': '敏感路径黑名单({n}): {v}',
  'policy.matrixTitle': '全角色矩阵:',
  'policy.matrixRow': '  {role} allow=[{allow}] approval=[{approval}] max=${max}',
  'quality.reportTitle': '===== 质量门禁报告 =====',
  'quality.fileResult': '{file}: {status}',
  'quality.pass': '✅ 通过',
  'quality.fail': '❌ 未通过',
  'quality.check': '  {mark} {name}: {value} {req}',
  'quality.req': '(要求: {threshold})',
  'quality.total': '总计: {passed}/{total} 通过',
};

/* ============================ 英文词条 ============================ */
const EN: Record<string, string> = {
  'app.product': 'fhcode',
  'app.tagline': 'Terminal AI coding agent (a Muse Code reimplementation)',
  'app.signature':
    'Jinjiang Feihongzhi Tech Enterprise Management Co., Ltd. · Feiyang Qiyuan R&D Center · Lead: Wu Cihong',
```

```
  'cli.brand': '[fhcode]',
  'cli.help': `fhcode (Feihong Code) v{version}

Usage:
  fhcode                              Enter interactive REPL
  fhcode "<request>"                 Run a single request
  fhcode --parallel "<request>"      Multi-subagent parallel mode (M2: git worktree isolation)
  fhcode /plan "<goal>"              Generate an implementation plan (read-only)
  fhcode /grill [path]               Red-team code review (read-only)
  fhcode /goal "<goal>"              Decompose and save a high-level goal
  fhcode sessions                     List historical sessions (M3 resume & audit)
  fhcode resume <id>                 Resume an interrupted session from checkpoint (M3)
  fhcode diff [<id>]                 Show session/workspace diff (M3)
  fhcode rollback <id> [--yes]       Roll back session changes (M3, destructive, needs --yes)

Enterprise (M4):
  fhcode whoami                       Current tenant / user / role / isolated dir / quota
  fhcode policy                       Show active RBAC policy & role matrix
  fhcode audit [--limit N]            View audit records (last 20 by default)
  fhcode audit verify                Verify audit hash-chain integrity
  fhcode tenants                      List all tenants & usage summary
  fhcode doctor                       Environment self-check (version/git/providers/paths/network)

Self-evolution (M6):
  fhcode model-stats                  View per-model performance stats
  fhcode experiences [path]           List the experience library

Web console (M5):
  fhcode serve [--port 8080]          Start the Web console (default http://localhost:8080)
                                     Token from FH_WEB_TOKEN or auto-generated (observe-only)

Autonomous coding (M8):
  fhcode code-write "<goal>"          Autonomous code writing (plan→write→test→review→fix)
  fhcode quality-gate [path]          Quality-gate review (security+quality+test coverage)
  fhcode self-improve                 Self-improvement stats & history

Autonomous SWE Agent (M9):
  fhcode swe "<goal>"                  Read repo→plan→implement+verify+self-heal→report
                                      --repo <path>  target repo (default cwd)
                                      --plan-only   plan only, no execution
                                      --verify-only verification only, no implementation
                                      --max-tasks N max subtasks (default 8)
                                      --max-retries N self-heal retries per task (default 2)

  fhcode --version                    Show version (-v)
  fhcode --help                       Show help (-h)
  fhcode --lang <zh|en>               Set UI language (Chinese / English)

Notes: offline mode is used automatically when FH_PROVIDERS is unset (scripted Mock drives the loop).
With FH_PROVIDERS (OpenAI-compatible / Ollama) configured, real LLMs run the tasks.
```

```
Enterprise mode is on by default (tenant isolation + RBAC + audit chain + quota); disable with FH_ENTERPRISE=false.
Signature: {signature}`,

  'repl.welcome': 'fhcode REPL (type a request and press Enter to run; type exit to quit)',
  'repl.hint': 'Hint: offline mode is used automatically when FH_PROVIDERS is not configured; /plan /grill /goal skills supported.\n',
  'repl.prompt': 'fhcode> ',
  'repl.errorPrefix': 'Error: ',
  'repl.unknownSkill': 'Unknown skill: /{cmd} (supported: /plan /grill /goal)',
  'repl.bye': '\nGoodbye.',
  'repl.stateReady': 'ready',
  'repl.stateRunning': 'running',

  'stream.toolCalling': 'Calling tools: {tools}',
  'stream.toolOk': 'ok',
  'stream.toolFail': 'failed',
  'stream.selfHeal': 'Self-heal retry: {category}',
  'stream.compact': 'Context compacted: {from} → {to} messages',
  'stream.done': 'Done · {iter} iterations · cost {cost}',

  'run.identity': 'Identity tenant={tenant} user={user} role={role}',
  'run.start': 'Task started (runId={id}{mode})',
  'run.resultTitle': '===== Result =====',
  'run.summary': 'Iterations: {iter} · Cost: {cost} · Log: {log}',
  'run.checkpoint':
    'Session checkpoint: {path} (manage with fhcode sessions / resume / diff)',
  'run.offlineFileYes': '(Offline demo file written: {file})',
  'run.offlineFileNo':
    '(No offline demo file produced; workspace: {cwd}. If blocked by policy, see fhcode audit for the reason)',
  'run.offlineFragment': ', offline mode',
  'run.modeOffline': 'offline',
  'run.modeLive': 'live',

  'run.parallelMode': '[fhcode] parallel mode: {offline}',
  'run.parallelResult': '===== Parallel Result =====',
  'run.parallelRepo': 'Repo root: {root} · Worktrees cleaned: {trees}',

  'run.noSessions': '(No historical sessions)',
  'run.sessionList': 'Sessions ({mode} mode, dir {home}):',
  'run.sessionItem':
    '- {id} | {status} | iter {iter} | cost {cost} | files {files} | {time}',
  'run.sessionGoal': '    goal: {goal}',

  'run.resumeDone': 'Session {id} already completed, no need to resume.',
  'run.resumeStart': '[fhcode] resuming session {id} (status: {status}, iterations: {iter})',

  'run.diffSession': '[fhcode] changes of session {id} (cwd={cwd}):',
  'run.diffCwd': '[fhcode] workspace changes in {cwd}:',

  'run.rollbackStart': '[fhcode] rolling back {n} files of session {id} (cwd={cwd})',
  'run.rollbackReverted': 'Reverted (tracked): {files}',
```

```
  'run.rollbackRemoved': 'Removed (untracked): {files}',
  'run.rollbackNote': 'Note: {errors}',

  'err.enterpriseDisabled':
    'Enterprise mode is disabled (FH_ENTERPRISE=false); this command is unavailable.',
  'err.runFailed': '[fhcode] run failed ({code}): {message}',
  'err.unexpected': 'Unexpected error: ',
  'err.hint':
    '(Detailed logs are in structured JSON output; for config errors check FH_PROVIDERS / .env)',

  'mode.offline': 'offline',
  'mode.live': 'live',

  'audit.empty': '(No audit records for tenant {tenant}; dir {dir})',
  'audit.header': 'Audit records {rows}/{all} (tenant {tenant}):',
  'audit.row':
    '#{seq} {ts} [{decision}] {action} by {user}({role}) run={run}',
  'audit.resource': '      resource: {resource}',
  'audit.reason': '      reason: {reason}',
  'audit.chainTail': 'Chain tail hash: {hash}…',
  'audit.verifyOk': '✅ Audit chain intact: {total} records, hash chain self-consistent and unaltered.',
  'audit.verifyFail': '❌ Audit chain verification failed: {total} records, broken at record #{brokenAt}',

  'tenants.empty': '(No tenant data yet; created automatically after one task)',
  'tenants.header': 'Tenant usage summary:',
  'tenants.tableHeader': '  TenantID            Sessions   TotalCost       Audits    LastActive',
  'tenants.row':
    '  {id} {sessions}   {cost}   {audit}   {last}',

  'modelStats.empty': '(No model performance data yet; generated automatically after tasks)',
  'modelStats.noRecords': '(No model stats recorded)',
  'modelStats.title': 'Model performance stats (M6):',
  'modelStats.tableHeader':
    '  ProviderID          Model               Calls  OK  Fail  Rate   AvgLat   TotalCost',

  'exp.empty': '(No experiences yet; accumulated automatically after tasks)',
  'exp.header': 'Experience library ({n} entries, source: {dir}):',
  'exp.tableHeader': '  ID                              Type               Title                  Rate  Uses',
  'exp.more': '  ... {n} entries total, showing first 10',

  'doctor.title': '===== fhcode doctor environment check =====',
  'doctor.node': 'Node version',
  'doctor.git': 'git availability',
  'doctor.gitMissing': 'git unavailable (diff/rollback/parallel mode will be limited)',
  'doctor.config': 'Model config',
  'doctor.configEmpty': 'No provider configured (offline mode will be used automatically)',
  'doctor.provider': 'Provider',
  'doctor.network': 'Network reachability',
  'doctor.networkOffline': 'Offline mode, skipping network probe',
  'doctor.home': 'Home dir writable',
```

```
  'doctor.sandbox': 'Sandbox mode',
  'doctor.sandboxUnavailable': '(config unavailable)',
  'doctor.docker': 'Docker sandbox (container mode)',
  'doctor.allOk': '✅ Environment ready, no issues',
  'doctor.issues': '⚠️ {n} issue(s) found, please fix them as suggested',

  'plugin.installUsage': 'Usage: fhcode plugin install <local-dir|git-url>',
  'plugin.installed': '✅ Plugin installed: {name} ({dir})',
  'plugin.installFailed': '❌ Plugin install failed: ',
  'plugin.empty': '(No plugins installed)',
  'plugin.listTitle': 'Installed plugins:',

  'skillMarket.localEmpty': '(No local skills installed)',
  'skillMarket.localTitle': 'Local skills ({n}):',
  'skillMarket.fetchFailed': '❌ Failed to fetch market index ({base}): ',
  'skillMarket.schemaWarn': '⚠️ Unrecognized market index schema ({schema}), treating as 0.1.0',
  'skillMarket.searchEmpty': '(No skills match "{q}" in the market)',
  'skillMarket.searchTitle': 'Market results for "{q}" ({n} found, source {base}):',
  'skillMarket.installHint': 'Install: fhcode skill-market install <skill-name> [--repo <market-url>]',
  'skillMarket.installUsage': 'Usage: fhcode skill-market install <skill-name> [--repo <market-url>]',
  'skillMarket.localSeed': '(network unreachable, using local seed templates/market/index.json)',
  'skillMarket.registered': '✅ auto-registered to local skill index (total {n} skills)',
  'skillMarket.notRegisteredWarn': '⚠️ installed but NOT discovered by local index: {name} (check SKILL.m
d format)',
  'skillMarket.notFound': '❌ Skill not found in market: {name}',
  'skillMarket.installed': '✅ Skill installed: {name} ({dir}), auto-discovered in tasks',
  'skillMarket.installFailed': '❌ Skill install failed: ',

  'team.start': 'Agent Team started ({mode}): shared task board + message bus',
  'team.reportTitle': '===== Agent Team Report =====',

  'serve.url': '[fhcode] Web console: {url}',
  'serve.token': '[fhcode] access token (FH_WEB_TOKEN): {token}',
  'serve.stop': '[fhcode] press Ctrl+C to stop',
  'serve.tokenAuto':
    '[fhcode] Web console token auto-generated (FH_WEB_TOKEN): {token}',
  'serve.started': '[fhcode] Web console started: http://localhost:{port}',

  'codewrite.resultTitle': '===== M8 Autonomous Write Result =====',
  'codewrite.files': 'Generated files: {files}',

  'quality.failed': '⚠️ {n} file(s) failed the quality gate, fix them before committing',

  'selfimp.title': '===== M6/M8 Self-Improvement System Status =====',
  'selfimp.reflections': 'Reflections: {n}',
  'selfimp.successRate': 'Success rate: {p}%',
  'selfimp.avgDuration': 'Avg duration: {ms}ms',
  'selfimp.expLib': 'Experience library: {n} unique experiences, cumulative weight {w}',
  'selfimp.topExp': 'Top experiences (by reuse count):',
  'selfimp.expItem': '  [{count}×] {type} | {title}',
  'selfimp.recent': 'Recent improvements:',
```

```
  'selfimp.record': '  {ts} | {ok} | patterns: {n}',
  'selfimp.improvement': '    → {imp}',
  'selfimp.noRecords': '(No improvement records yet; generated automatically after tasks)',
  'selfimp.learnPreview': '----- Learned-prompt preview (goal: {goal}) -----',
  'selfimp.noLearned': '(No experience available to inject)',

  'swe.noProvider':
    '[fhcode] No real model provider configured; cannot enter live mode. Connect via one of:\n' +
    '  1) Env vars: FH_MODEL_NAME=<model> FH_MODEL_TYPE=ollama|openai-compatible FH_MODEL_BASE_URL=<url> [FH_MODEL_API_KEY=<token>]\n' +
    '  2) Config file: models.providers array in ./fhcode.config.json\n' +
    '  3) FH_PROVIDERS env var (JSON array)\n' +
    'Local Ollama example: FH_MODEL_NAME=qwen2.5-coder:1.5b FH_MODEL_TYPE=ollama fhcode swe "..."',
  'swe.start': '[fhcode] launching autonomous SWE agent ({offline}, repo={cwd})',
  'swe.reportTitle': '===== Autonomous SWE Agent Report =====',
  'harness.start': '[fhcode] starting evaluation harness ({mode} mode · split={split} · limit={limit})',
  'harness.noProvider': '[fhcode] No real model provider configured (set FH_PROVIDERS or FH_MODEL_NAME, then use --mode real)',
  'harness.reportWritten': 'Report written to: {path}',
  'harness.summary': 'Summary: {completed}/{total} passed ({rate}%)',

  'approve.prompt': '[approve] allow execution: {action}\n  type y/yes to allow, anything else to deny: ',

  'enterprise.whoamiTitle': 'Current identity (enterprise mode)',
  'enterprise.tenantId': 'Tenant ID',
  'enterprise.userId': 'User ID',
  'enterprise.role': 'Role',
  'enterprise.sessionDir': 'Isolated dir',
  'enterprise.quota': 'Quota',
  'enterprise.quotaUsed': 'used',
  'enterprise.quotaLimit': 'limit',
  'enterprise.policyTitle': 'Active policy & role matrix',
  'enterprise.roleColumn': 'Role',
  'enterprise.permissionsColumn': 'Permissions',
  'enterprise.unlimited': 'unlimited',
  'enterprise.unknown': 'unknown',
  'enterprise.on': 'on',
  'enterprise.off': 'off (FH_ENTERPRISE=false)',
  'enterprise.none': 'none',
  'enterprise.whoami.tenant': 'Tenant: {v}',
  'enterprise.whoami.user': 'User  : {v}',
  'enterprise.whoami.role': 'Role  : {v}',
  'enterprise.whoami.isoDir': 'Isolated dir : {v}',
  'enterprise.whoami.session': '  session     : {v}',
  'enterprise.whoami.audit': '  audit       : {v}',
  'enterprise.whoami.goal': '  goal        : {v}',
  'enterprise.whoami.taskCap': 'Per-task cap : {v}',
  'enterprise.whoami.used': 'Used today   : {v}',
  'enterprise.whoami.auditCount': 'Audit count : {v}',
  'enterprise.whoami.mode': 'Enterprise   : {v}',
  'policy.version': 'Policy version: v{v}',
  'policy.role': 'Current role: {v}',
```

```
  'policy.allow': '  allow: {v}',
  'policy.approval': '  approval: {v}',
  'policy.taskCap': 'Per-task cost cap: {v}',
  'policy.tenantCap': 'Tenant daily cap: {v}',
  'policy.denyShell': 'Dangerous shell blacklist ({n}): {v}',
  'policy.denyPaths': 'Sensitive path blacklist ({n}): {v}',
  'policy.matrixTitle': 'Full role matrix:',
  'policy.matrixRow': '  {role} allow=[{allow}] approval=[{approval}] max=${max}',
  'quality.reportTitle': '===== Quality Gate Report =====',
  'quality.fileResult': '{file}: {status}',
  'quality.pass': '✅ passed',
  'quality.fail': '❌ failed',
  'quality.check': '  {mark} {name}: {value} {req}',
  'quality.req': '(required: {threshold})',
  'quality.total': 'Total: {passed}/{total} passed',
};

/**
 * 取当前语言字符串并做 {name} 插值。
 * 缺失 key 时回退中文，再回退 key 本身，保证永不崩溃。
 */
export function t(key: string, params?: Record<string, string | number>): string {
  const table = currentLang === 'en' ? EN : ZH;
  let s: string | undefined = table[key];
  if (s == null) s = ZH[key];
  if (s == null) return key;
  if (params) {
    for (const k of Object.keys(params)) {
      s = s.replace(new RegExp(`\\{${k}\\}`, 'g'), String(params[k]));
    }
  }
  return s;
}

/** 便捷：返回当前语言下的品牌标签（如 [飞虹 Code] / [fhcode]） */
export function brandTag(): string {
  return t('cli.brand');
}

```

┌── src/shared/types.ts ──┐

```
/**
 * 飞虹 Code (对标 Muse Code · 自研内核)
 * 晋江市飞虹智科技企业管理有限公司 · 飞扬企源研发中心 · 负责人：吴赐虹
 *
 * 共享类型定义
 */

/** 一次 CLI 运行 / 一个任务的唯一标识 */
export type RunId = string;

```